Chapter 17 — Interview Question Bank

How to use this bank

This chapter collects the React questions that come up again and again in real interviews: the ones repeated across the big public GitHub question collections, popular blog and community question lists, and the machine-coding prompts reported by candidates (the sources are listed at the end). The questions are worded the way interviewers usually say them, so the wording itself becomes familiar.

- ⋆ marks a question that appears in many of those sources — learn these first.
- Every question has a **short answer you can say out loud** in 20–60 seconds. Where this book teaches the topic in depth, a → **More** link takes you to the full explanation (with runnable code and real output). Questions the book does not cover elsewhere still get a complete short answer here, so nothing is missing.
- Chapter 15 already has 53 interview questions with 30-second answers and 8 machine-coding builds with tests. This bank **links to those** instead of repeating them in full, and adds every common question they don't include. ("Ch 15 Q20" means question 20 in Chapter 15.)
- Everything is written for **React 19** (the book uses 19.3), React Router 8, Redux Toolkit 2, Zustand 5, TanStack Query 5 and Next.js 16. Answers that mention older APIs say so.
- **How to drill:** cover the answer, say yours out loud (a full answer: *what it is → why → an example → a trade-off*), then uncover and compare. Mark the ones you stumbled on and repeat them the next day. For the coding tasks in the last groups, write the code on a blank file with a timer, then compare with the linked build.

Terms are defined the first time they appear. A few that appear everywhere:

- **Component** — a JavaScript function that returns a description of UI (JSX). **Props** — the inputs a parent passes to a component. **State** — data a component remembers between renders and that, when changed, makes React re-render.
- **Render** — React calling your component function to get the new UI description. **Commit** — React applying the differences to the real DOM (the browser's tree of page elements).
- **Hook** — a function whose name starts with `use` that lets a component use a React feature (state, effects, context …).

---

1. React fundamentals and JSX

Q1. ⋆ What is React, and what are its main features? React is a JavaScript **library** for building user interfaces out of **components**. Its main features: **declarative** UI (you describe what the screen should look like for the current data, and React works out the DOM changes), **component-based** reuse, **one-way data flow** (data goes down through props), a **virtual DOM / reconciliation** step that updates only what changed, Hooks for state and side effects, and the ability to render on the server (SSR, Server Components) as well as in the browser. It is "just the view layer": routing, data fetching and build tooling come from frameworks or libraries. → *More:* §1.1 What React is · §1.2 Declarative UI

Q2. ⋆ Is React a library or a framework? What are its advantages and limitations? React itself is a library — it renders UI and manages component state, but leaves routing, data loading and the build to you (or to a framework such as Next.js or React Router's framework mode). Advantages: a small, stable core idea (UI = f(state)), huge ecosystem, reusable components, same skills on web, server and mobile (React Native). Limitations: many decisions to make (router, state library, styling), JSX and build tools to learn, client-only apps send a lot of JavaScript and are poor for SEO unless you add SSR, and performance tuning (re-renders) needs understanding. → *More:* §1.1 What React is

Q3. What is declarative vs imperative UI programming? Why does React choose declarative? Imperative code lists the steps: "find the button, add a class, change the text, remove the spinner". **Declarative** code describes the result: "when `loading` is true, show a spinner; otherwise show the list". React is declarative: you return the UI for the current state, and React computes the DOM operations. This

removes a whole class of bugs where the DOM and the data get out of sync, and makes components easy to read and test. → *More:* §1.2 Declarative UI

Q4. ⋆ What is JSX? Do browsers understand it? **JSX** is a syntax extension to JavaScript that looks like HTML and describes UI: `<h1 className="t">Hi {name}</h1>`. Browsers do **not** understand it; a compiler (Babel, esbuild, SWC, TypeScript) turns each tag into a function call — with the modern "automatic runtime" that is `jsxs('h1', { className: 't', children: ['Hi ', name] })` (`jsx` for a single child), imported from `react/jsx-runtime`. The result is a plain object called a **React element**. JSX is optional but almost everyone uses it. → *More:* §1.8 JSX

Q5. What are the rules of JSX? (1) Return **one root** element — wrap siblings in a `<div>` or a Fragment `<>…</>`. (2) **Close every tag**, including `<img />` and `<br />`. (3) Use **camelCase** attribute names: `className`, `htmlFor`, `onClick`, `tabIndex` (but `aria-*` and `data-*` keep their dashes). (4) Curly braces `{}` hold any JavaScript **expression** (not statements like `if` or `for`). (5) `style` takes an object: `style={{ fontSize: 14 }}`. (6) Component names start with a **capital letter**. → *More:* §1.8 JSX · §16.1 JSX rules

Q6. Why must JSX return a single root element? What is a Fragment and why is it better than an extra `<div>`? A component returns one value, and JSX compiles to one function call, so two sibling tags would be two values. A **Fragment** (`<>…</>` or `<Fragment>…</Fragment>`) groups children without adding a DOM node. That keeps the DOM smaller and avoids breaking layouts that depend on direct children (CSS grid/flex, `<table>` rows, `<ul>` items). Use the long form `<Fragment key={id}>` when you need a key in a list. → *More:* §1.8 JSX · §2.8 Rendering lists and keys

Q7. Why does React use `className` instead of `class`, and `htmlFor` instead of `for`? JSX is JavaScript, and `class` and `for` are reserved words in JavaScript. React's props follow the DOM **property** names (`element.className`, `label.htmlFor`), not the HTML attribute names. (On a built-in element, writing `class` triggers a development warning; on custom elements — web components — React 19 passes `class` through as an attribute.) → *More:* §1.8 JSX

Q8. ⋆ What is the difference between a React element and a React component? (And a React node?) A **component** is the function (or class) you write — a recipe. An **element** is the plain, immutable object a component call produces from JSX, like `{ type: 'button', props: { children: 'Save' } }` — a description of what to show, not the DOM node itself. A **React node** is anything React can render: an element, a string, a number, an array of nodes, `null`, `undefined` or a boolean (the last three render nothing). → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · §2.1 Function components

Q9. How do you write comments, loops and conditions in JSX? Comments go inside braces: `{/* note */}`. Loops use `array.map()` returning elements with a `key`: `{items.map(i => <li key={i.id}>{i.name}</li>)}`. Conditions use expressions: the ternary `cond ? <A/> : <B/>`, `cond && <A/>`, or an early `return` / a variable computed with `if` above the JSX. Statements (`if`, `for`) can't appear directly inside `{}` because braces need an expression. → *More:* §2.7 Conditional rendering · §2.8 Rendering lists and keys

Q10. How does JSX protect against XSS (cross-site scripting) attacks? What is `dangerouslySetInnerHTML`? **XSS** is an attack where untrusted text is injected into a page as HTML/script and executed. React **escapes** every value you put in `{}` — it is inserted as text, so `<script>` shows as characters. `dangerouslySetInnerHTML={{ __html: html }}` bypasses this and inserts raw HTML; the long name is a warning. Use it only with HTML you trust or have **sanitised** (cleaned with a library like DOMPurify). Also watch `href={userUrl}` — recent React versions block `javascript:` URLs there, but validate user-supplied URLs anyway. → *More:* §1.8 JSX

Q11. What is the "new JSX transform"? Do you still need `import React from 'react'`? Since React 17, compilers use the **automatic runtime**: they insert `import { jsx } from 'react/jsx-runtime'` themselves, so files with JSX no longer need `import React`. It also gives slightly smaller output. You still import Hooks and APIs you use (`import { useState } from 'react'`). → *More:* §1.8 JSX

Q12. What is the difference between `react` and `react-dom`? Why are they separate packages? `react` holds the component model: elements, Hooks, Context, `memo`, `lazy`. `react-dom` is a **renderer** — it connects that model to the browser DOM (`react-dom/client` with `createRoot`/`hydrateRoot`, `createPortal`, `flushSync`, `useFormStatus`) and to HTML on the server (`react-dom/server`, `react-dom/static`). Other renderers (React Native, react-three-fiber, test renderers) reuse `react` with a different target. → *More:* §1.5 Setting up a React project · §13.2 The server rendering APIs

Q13. How do you mount a React app in React 19? What happened to `ReactDOM.render`? `createRoot(document.getElementById('root')).render(<App />)` from `react-dom/client`; for server-rendered HTML use `hydrateRoot(container, <App />)`. `ReactDOM.render` and `ReactDOM.hydrate` were deprecated in React 18 and **removed in React 19**, along with `unmountComponentAtNode` (use `root.unmount()`) and `findDOMNode` (use refs). → *More:* §1.7 Your first component · §13.4 Hydration end to end

Q14. What is `<StrictMode>` and what does it do? `<StrictMode>` is a development-only wrapper that helps find bugs. Inside it, React renders components **twice**, runs Effect **setup → cleanup → setup** once more on mount, runs ref callbacks twice, and warns about deprecated APIs. Pure components and symmetric cleanups aren't affected; impure ones show their bugs immediately. It does nothing in production. → *More:* §5.4 StrictMode's double run (development only) · Ch 15 Q19

Q15. What changed in React 18? (Common "what's new" question.) React 18 introduced **concurrent rendering** (React can prepare an update in the background and interrupt it for a more urgent one), the new `createRoot` API, **automatic batching** of state updates everywhere (timeouts, promises, native handlers), **transitions** (`useTransition`, `startTransition`) and `useDeferredValue`, **Suspense on the server** with streaming SSR and selective hydration, and new Hooks `useId`, `useSyncExternalStore` and `useInsertionEffect`. → *More:* §3.4 Batching · §7.10 `useTransition` — non-urgent updates · §13.3 Streaming SSR with `<Suspense>`

Q16. ⋆ What's new in React 19? Actions (async functions in transitions, with pending state handled for you), `<form action={fn}>`, the Hooks `useActionState`, `useFormStatus` and `useOptimistic`, the `use()` API for reading promises and context, **Server Components and Server Functions** as stable features, **`ref` as a normal prop** (no `forwardRef`), ref callback **cleanup** functions, `<Context value>` as a provider, built-in `<title>` / `<meta>` / `<link>` hoisting, stylesheet and script support, resource preloading (`preload`, `preinit`), better hydration error messages, and full custom-element support. 19.2 added `<Activity>` and `useEffectEvent`. Removed: `propTypes` and function `defaultProps`, string refs, legacy context, `ReactDOM.render`. → *More:* §4.4 Form Actions — `<form action={fn}>` · §5.10 Passing refs to your own components · §9.11 Suspense and `use()` · §13.7 React Server Components

Q17. What is Create React App's status? How would you start a React project today? Create React App was **deprecated** (February 2025) — it is unmaintained. react.dev recommends starting with a **framework** (Next.js, React Router framework mode, Expo for mobile) or, for a client-only app, a **build tool** like **Vite** (`npm create vite@latest`). Frameworks bring routing, data loading and server rendering; Vite gives a fast dev server and bundling only. → *More:* §1.5 Setting up a React project

Q18. Should you still learn class components? How do lifecycle methods map to Hooks? New code uses function components, but you will meet classes in old code — and error boundaries still need one. Rough mapping: `constructor` / `this.state` → `useState`; `componentDidMount` + `componentDidUpdate` + `componentWillUnmount` → one `useEffect` with setup, dependencies and cleanup (think in "synchronise with X", not lifecycle events); `shouldComponentUpdate` / `PureComponent` → `memo`; `getDerivedStateFromProps` → compute during render or reset with `key`; `componentDidCatch` → still a class (error boundary). → *More:* §5.2 `useEffect` basics · §14.9 Error boundaries

---

2. Components, props and composition

Q19. ⋆ What is the difference between state and props? Props are inputs passed **from the parent**; the component must treat them as read-only. **State** is data the component **owns** and changes with a setter, which triggers a re-render. A value that is state in a parent is often a prop in the child. Rule of thumb: if it can be computed from props or other state, it is neither — compute it during render. → *More:* §2.2 Props · §3.2 State: a component's memory — `useState`

Q20. ⋆ Why are props read-only? What happens if a child mutates a prop? React's model is that a component is a **pure function of its props and state**: same inputs, same output. Props belong to the parent; if a child changes them, the parent doesn't know, nothing re-renders, and the UI shows stale or inconsistent data (and other components sharing that object see surprise changes). To change something, the parent passes a callback (`onChange`) and updates its own state. → *More:* §2.2 Props · §2.5 Pure components

Q21. What is the `children` prop? Whatever you put between a component's opening and closing tags arrives as `props.children`: `<Card><h2>Title</h2></Card>`. It lets you build wrappers (cards, layouts,

modals) that don't need to know their content. `children` can be any React node — text, elements, arrays, or even a function (the render-prop pattern). → *More:* §2.3 The `children` prop

Q22. ⋆ What is prop drilling, and how do you avoid it? **Prop drilling** is passing a prop through several layers of components that don't use it, just to reach a deep child. It makes code noisy and refactors painful. Fixes, in order: **composition** (pass the deep component itself as `children` or a slot prop so the middle layers don't see the data), **Context** for values many components need (theme, current user, locale), or a **store** (Zustand, Redux) for frequently changing shared state. → *More:* §2.4 Composition · §6.3 Context — pass data deeply without props

Q23. How do you give props default values in React 19? Use JavaScript **default parameters** in destructuring: `function Button({ size = 'md', children })`. `Component.defaultProps` for function components was **removed in React 19** (it still works for classes). Defaults apply only when the prop is `undefined`, not when it is `null`. → *More:* §2.2 Props

Q24. How do you validate props? What happened to PropTypes? React 19 **removed** the built-in `propTypes` checks — they are silently ignored. Use **TypeScript** (checked at build time, and in your editor) for prop types; validate **runtime data** (API responses, form input) with a schema library such as **Zod**. → *More:* §11.2 Typing props

Q25. What is a pure component? What is `PureComponent`? A **pure** component returns the same JSX for the same props, state and context, and doesn't change anything outside itself while rendering (no mutating variables created before the render, no fetching, no DOM writes). React relies on this to render twice, pause or skip renders safely. `PureComponent` is a legacy **class** base that skipped re-rendering when props and state were shallowly equal — the function equivalent is `memo`. → *More:* §2.5 Pure components · §10.4 `memo` — skipping a child's re-render

Q26. What is composition, and why does React prefer it over inheritance? **Composition** means building components by combining others — passing JSX as `children` or as named "slot" props (`<Layout sidebar={<Nav/>}>`). React has no need for component inheritance: anything a base class would share can be a component you wrap, a prop, or a custom Hook. Composition keeps components decoupled and avoids deep, fragile class hierarchies. → *More:* §2.4 Composition · §14.1 Composition first

Q27. ⋆ How do you conditionally render content? Which patterns are there? Early `return null` (render nothing), `if` statements assigning JSX to a variable, the ternary `a ? <X/> : <Y/>`, and `cond && <X/>`. Watch the `&&` trap: `{count && <Badge/>}` renders `0` when `count` is 0, because `0` is a renderable number — write `{count > 0 && <Badge/>}`. For many cases, use an object map: `const views = { loading: <Spinner/>, error: <Err/> }; return views[status];`. → *More:* §2.7 Conditional rendering · Ch 15 Q7

Q28. How do you pass data from a child to a parent? Between siblings? Child → parent: the parent passes a **callback** prop, and the child calls it with data (`onSelect(item)`). Siblings: **lift the state up** to their closest common parent, which passes the value down to one and the setter (or a handler) down to the other. For distant components, use Context or a store. → *More:* §3.7 Lifting state up

Q29. What does "lifting state up" mean? When two components need the same changing data, you move that state to their **closest common parent** and pass it down as props, with callbacks to change it. There is then a single **source of truth** (one place that owns the data), so the two can't disagree. → *More:* §3.7 Lifting state up

Q30. What is one-way (unidirectional) data flow, and why is it useful? Data flows **down** the tree through props; changes flow **up** only as function calls (events) that update state at its owner, which then re-renders downwards. Because every piece of state has one owner and one way to change, you can trace any value to where it came from — easier debugging than two-way binding, where a change can come from anywhere. → *More:* §3.7 Lifting state up · §2.2 Props

Q31. Why must component names start with a capital letter? JSX uses the case of the tag to decide what to create: lowercase (`<div>`, `<my-widget>`) means a built-in DOM element or custom element given as a **string**; capitalised (`<MyWidget>`) means a **variable** holding a component. `<myWidget/>` would try to create an unknown HTML tag. → *More:* §1.7 Your first component

Q32. ⋆ Why should you never define a component inside another component? A component defined inside another's body is a **new function on every render**. React compares component types by identity, so it sees a different type each time, **unmounts the old subtree and mounts a fresh one** — all state inside is lost (inputs lose focus and text) and it's slow. Define components at the top level of the module and

pass data via props. → *More:* §3.10 State is tied to a position in the tree · §10.10 Common performance bugs

Q33. What is the spread-props pattern (`{...props}`), and what's its risk? `<input {...rest} />` forwards all remaining props, handy for wrapper components that extend native elements. Risks: you can pass unknown props to DOM elements (React warns about invalid attributes), accidentally override props you set earlier (order matters: later wins), and make the component's API unclear. Destructure what you use and spread only the rest, and type it with `ComponentProps<'input'>`. → *More:* §2.2 Props · §11.3 Typing children and extending native elements

Q34. What is a "controlled component" in general (not only inputs)? A component is **controlled** when its important state is passed in by the parent as a prop together with a change callback (`value` + `onChange`, `open` + `onOpenChange`); it is **uncontrolled** when it keeps that state itself (maybe with a `defaultValue`). Controlled gives the parent full power (validate, sync, reset); uncontrolled is simpler to use. Good reusable components often support both. → *More:* §4.2 Controlled vs uncontrolled inputs · §14.4 Compound components

---

3. State and events

Q35. ⋆ What is `useState`, and why does it return an array? `const [value, setValue] = useState(initial)` gives a component a piece of memory that survives re-renders, plus a function to change it and schedule a re-render. It returns an **array** (a pair) so you can name both parts freely with array destructuring — `[count, setCount]`, `[user, setUser]` — which an object return wouldn't allow as neatly. → *More:* §3.2 State: a component's memory — `useState`

Q36. ⋆ Is `setState` synchronous or asynchronous? Why doesn't the value change right after I call the setter? The setter **schedules** an update; it doesn't change the variable you are holding. Your `count` is a **snapshot** — a constant for this render. React re-runs the component later (after the event handler finishes) and that new render sees the new value. So `setCount(5); console.log(count)` logs the old value. If you need the next value, compute it in a variable first, or read it on the next render / in an Effect. → *More:* §3.3 State as a snapshot · Ch 15 Q1

Q37. ⋆ What is the functional (updater) form of `setState`, and when should you use it? `setCount(c => c + 1)` passes a function that receives the **latest pending value** and returns the next one. React queues updaters and applies them in order, so three calls add 3, whereas `setCount(count + 1)` three times adds only 1 (all read the same snapshot). Use it whenever the next state depends on the previous state — especially in async code, intervals and multiple updates in one event. → *More:* §3.5 Updater functions — queueing updates · Ch 15 Q8

Q38. ⋆ What is batching? Can you opt out? Batching means React groups several state updates made during the same event into **one** re-render. Since React 18 this is **automatic everywhere** — event handlers, timeouts, promises, native listeners. To force an immediate DOM update (rare, e.g. to scroll to an element you just added), wrap the update in `flushSync(() => setX(...))` from `react-dom`. → *More:* §3.4 Batching · Ch 15 Q9

Q39. What does `useState(() => compute())` (lazy initial state) do? Passing a **function** makes React call it **only on the first render** to get the initial value. Use it when the initial value is expensive — reading `localStorage`, parsing a big object. `useState(compute())` (calling it) would run the expensive work on every render and throw the result away. → *More:* §3.2 State: a component's memory — `useState` · §7.4 `useLocalStorage` — state that survives a reload

Q40. ⋆ How do you update an object or array in state? Why not mutate it? Create a **new** object/array: `setUser({ ...user, name })`, `setItems([...items, item])`, `items.filter(...)` to remove, `items.map(...)` to change one, `toSorted()` / `toReversed()` instead of `sort` / `reverse`. React compares old and new state with `Object.is` (reference equality); if you mutate and pass the same reference, React thinks **nothing changed** and skips the render. Mutation also breaks `memo`, Effects dependencies, undo/redo and DevTools history. → *More:* §3.6 Updating objects and arrays immutably · Ch 15 Q10

Q41. How do you update nested state? What is Immer? Copy every level on the path to the change: `setForm({ ...form, address: { ...form.address, city } })`. If this gets deep, **flatten** the state (store items by ID) or use **Immer**, a library that lets you write "mutating" code on a draft (`draft.address.city =`

`city`) and produces a new immutable object for you (`useImmer`, and Redux Toolkit's `createSlice` uses it internally). → *More:* §3.6 Updating objects and arrays immutably · §6.7 Redux Toolkit

Q42. Which array methods are safe for state, and which mutate? Mutating (avoid on state): `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`, assignment `arr[i] = x`. Non-mutating (use): spread `[...a, x]`, `concat`, `filter`, `map`, `slice`, and ES2023's `toSorted`, `toReversed`, `toSpliced`, `with(i, x)`. If you must use `sort`, copy first: `[...a].sort()`. → *More:* §3.6 Updating objects and arrays immutably

Q43. What is derived state, and why is copying props into state a bug? Derived values can be calculated from props or state (a filtered list, a full name, a total). Store the minimal state and compute the rest during render. Copying a prop into state (`useState(props.value)`) only uses the prop's **first** value — later changes are ignored, so the UI gets out of sync. If you really need to reset internal state when a prop changes, give the component a `key`. → *More:* §3.8 Derived state — choosing the minimal state · Ch 15 Q11

Q44. ⋆ How are React events different from DOM events? What is a SyntheticEvent? React handlers are camelCase props (`onClick`) that take a **function**, not a string. React passes a **SyntheticEvent** — a wrapper with the same interface across browsers (`e.target`, `e.preventDefault()`, `e.stopPropagation()`), the native one at `e.nativeEvent`. React attaches listeners at the **root container** and dispatches them itself (event delegation). You can't `return false` to prevent default — call `e.preventDefault()`. Since React 17 events aren't pooled, so reading `e` asynchronously is fine. → *More:* §3.1 Responding to events · Ch 15 Q15

Q45. `onClick={handleClick}` vs `onClick={handleClick()}` vs `onClick={() => handleClick(id)}`? The first passes the function — correct. The second **calls** it during render and passes its return value (often `undefined`); if it sets state, you get an infinite loop ("Too many re-renders"). The third passes a new arrow function that calls the handler with an argument when clicked — the normal way to pass arguments. → *More:* §3.1 Responding to events

Q46. What are capture-phase events in React? DOM events travel **down** from the root to the target (capture phase), then **up** back to the root (bubble phase). React handlers run in the bubble phase by default; add `Capture` to the name — `onClickCapture` — to run during capture, which fires before children's handlers (useful for analytics or intercepting). → *More:* §3.1 Responding to events

Q47. How do you reset a component's state? Change its `key`: `<Profile key={userId} />`. A different key tells React it's a different component, so it unmounts the old one and mounts a fresh one with initial state. Alternatives: lift the state up and reset it in the parent, or call the setters with initial values (for a "Reset" button). → *More:* §3.10 State is tied to a position in the tree · Ch 15 Q5

Q48. Where should state live? How do you decide the minimal state? Put state in the **lowest common parent** of the components that use it. Keep it **minimal**: don't store what you can compute, don't duplicate the same data in two places (store an ID, not a copy of the selected object), and don't use state for values that never change or don't affect rendering (use a constant or a ref). Server data belongs in a data-fetching cache, URL-shareable state in the URL. → *More:* §3.8 Derived state — choosing the minimal state · §6.1 Kinds of state — and where each belongs · Ch 15 Q12

Q49. If I set state to the same value, does the component re-render? React compares with `Object.is`. If the new value is the same, React **bails out** — it won't re-render the children or run Effects. (It may still call the component function once before deciding, so don't rely on the body not running.) This is another reason mutation fails: same reference = "no change". → *More:* §10.2 When components re-render

Q50. What does "state is tied to a position in the tree" mean? React keeps a component's state attached to **where** it sits in the rendered tree, not to the JSX variable. Render the same component type at the same position and state is kept; change the type at that position (or remove it) and the state is destroyed. That's why `{isA ? <Counter/> : <Counter/>}` keeps the count (same position, same type) unless you give them different keys. → *More:* §3.10 State is tied to a position in the tree

---

4. Rendering, reconciliation, keys and the virtual DOM

Q51. ⋆ What is the virtual DOM, and how does it work? The "virtual DOM" is React's in-memory description of the UI — the tree of elements your components return. When state changes, React renders a new tree, **compares** it with the previous one (reconciliation), and applies only the differences to the

real DOM. DOM work is the slow part, so doing only the minimal set of changes, batched, is efficient — and you get to write UI as if you re-rendered everything. React's own docs now talk about "render and commit" rather than "virtual DOM", but interviewers still use the term. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · Ch 15 Q3

Q52. ⋆ What is reconciliation, and what is the diffing algorithm's main idea? Reconciliation is how React decides what changed between two renders. A general tree-diff costs $O(n^3)$; React uses an $O(n)$ **heuristic** with two assumptions: (1) elements of a **different type** produce different trees — React destroys the old subtree and builds a new one; (2) among siblings, **keys** identify which child is which across renders. Same type at the same position → React keeps the DOM node/component and updates only changed props. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · §2.8 Rendering lists and keys

Q53. Virtual DOM vs real DOM vs Shadow DOM? The **real DOM** is the browser's live tree of page elements. The **virtual DOM** is React's JavaScript description of it, used to compute changes. The **Shadow DOM** is an unrelated browser feature for **web components**: it attaches a hidden, encapsulated DOM subtree to an element so its styles and markup don't leak in or out. React can render custom elements that use Shadow DOM. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief)

Q54. Is the virtual DOM "faster than the DOM"? Not by itself — the diff is extra work on top of the DOM updates. Its value is that it lets you write simple declarative code ("re-render everything") while React keeps actual DOM writes small and batched. Frameworks without a virtual DOM (Svelte, Solid) use compile-time or fine-grained reactivity and can be faster in benchmarks. For React apps, slowness usually comes from too many re-renders or too much DOM, not from the diff itself. → *More:* §1.3 Rendering, the virtual DOM and reconciliation (brief) · §10.1 The render cycle: trigger → render → commit

Q55. ⋆ What is React Fiber? Fiber is the reconciler (the core algorithm) React has used since v16. Each component instance has a "fiber" — a unit of work in a linked tree — so React can **split rendering into small chunks, pause, prioritise, and resume** instead of doing one long blocking pass. That is what makes concurrent features possible (transitions, Suspense, `useDeferredValue`, selective hydration). Fibers also store each component's Hook state. → *More:* §7.1 What a Hook really is · §10.1 The render cycle: trigger → render → commit

Q56. ⋆ Why do lists need keys? Why is the array index a bad key? Keys tell React which item is which between renders, so it can keep the right DOM nodes and state when items are added, removed or reordered. With the **index** as key, inserting at the top or sorting shifts every key: React reuses the wrong components, so input text, checkbox state and focus jump to the wrong rows, and it updates every row. Use a stable, unique ID from the data (`item.id`); generate IDs when items are created, never during render (`Math.random()` as a key remounts everything every time). → *More:* §2.8 Rendering lists and keys · Ch 15 Q4

Q57. When is it safe to use the index as a key? When the list is **static**: it is never reordered, filtered or inserted into (other than appending at the end), and items have no state or uncontrolled inputs. Example: a fixed list of labels. If any of these might change, use real IDs. → *More:* §2.8 Rendering lists and keys

Q58. Do keys need to be globally unique? Can you read `props.key`? Keys must be unique only among **siblings** in the same list; two different lists can reuse keys. `key` (like `ref` in older versions) is consumed by React and **not passed** to the component as a prop — pass the ID again under another name if the child needs it. → *More:* §2.8 Rendering lists and keys

Q59. Can you use keys outside lists? Yes — a `key` on any element forces React to treat it as a different component when the key changes. `<ChatRoom key={roomId} />` remounts the room (fresh state, fresh Effects) when the room changes. This is the clean way to reset state on an identity change. → *More:* §3.10 State is tied to a position in the tree · Ch 15 Q5

Q60. ⋆ When does a component re-render? When (1) its **state** changes, (2) its **parent re-renders** (by default all children re-render, whether props changed or not), (3) a **context** it reads changes, or (4) an external store it subscribes to (via `useSyncExternalStore` / a store hook) changes. Props changing is not a separate trigger — props change only because the parent re-rendered. `memo` can skip case (2) when props are equal. → *More:* §10.2 When components re-render · Ch 15 Q6

Q61. ⋆ What are the render and commit phases? Which must be pure? **Render**: React calls components to compute the new UI — must be **pure** (no side effects), because React may call it twice, pause it, or throw it away. **Commit**: React applies the DOM changes, sets refs, runs `useLayoutEffect`, the browser paints, then `useEffect` runs. Side effects go in event handlers (first choice) or Effects. → *More:* §10.1 The render cycle: trigger → render → commit · Ch 15 Q2

Q62. Does re-rendering mean the DOM is updated? No. A **re-render** only means React called your component again. If the resulting JSX is the same, the commit does nothing to the DOM. Re-renders cost CPU time, but they are usually cheap; they become a problem when components are expensive or there are very many. → *More:* §10.1 The render cycle: trigger → render → commit

Q63. What happens when an element changes type, e.g. `<div>` to `<section>`, or `<A/>` to `<B/>` at the same position? React treats it as a different tree: it **unmounts** the old element and everything beneath it (running cleanups, destroying state and DOM nodes) and mounts the new one from scratch. This is why wrapping a subtree conditionally (`{wrap ? <div><Form/></div> : <Form/>}`) resets the form's state. → *More:* §3.10 State is tied to a position in the tree · §1.3 Rendering, the virtual DOM and reconciliation (brief)

Q64. What is concurrent rendering? In concurrent rendering React can work on a render **without blocking** the main thread: it can start rendering a low-priority update, pause to handle a click or keystroke, and then continue or restart. You opt in per update with transitions (`startTransition`, `useTransition`, `useDeferredValue`) or Suspense. Urgent updates (typing) stay responsive while big updates (filtering a long list) happen in the background. → *More:* §7.10 `useTransition` — non-urgent updates · §7.11 `useDeferredValue` — let part of the UI lag behind

---

5. `useEffect`, its pitfalls, and refs

Q65. ⋆ What is `useEffect`, and what is it for? `useEffect(setup, deps)` runs code **after React has committed** the render to the screen, to **synchronise the component with something outside React**: a subscription, a WebSocket, a timer, a non-React widget, `document.title`, or (in simple apps) a fetch. The setup may return a **cleanup** function that undoes it. It is not the place for logic that responds to a user action (that belongs in the event handler) or for computing values from state (do that during render). → *More:* §5.2 `useEffect` basics · Ch 15 Q16

Q66. ⋆ What do no dependency array, `[]`, and `[a, b]` mean? No array: the Effect runs after **every** render. `[]`: only after the **first** mount (and its cleanup on unmount). `[a, b]`: after mount and whenever `a` or `b` changed since the last render (compared with `Object.is`). The array is not a "choose when to run" switch — it must list **every reactive value** (props, state, and values computed from them) the Effect reads; the `react-hooks/exhaustive-deps` lint rule checks that. → *More:* §5.5 Dependencies in depth · Ch 15 Q17

Q67. ⋆ When does the cleanup function run? Before the Effect runs again (with the **old** values, before the next setup), and once when the component **unmounts**. In development with StrictMode it also runs once right after the first mount (setup → cleanup → setup) to prove the cleanup is correct. Typical cleanups: `clearInterval`, `removeEventListener`, `unsubscribe`, `controller.abort()`, `socket.close()`. → *More:* §5.3 Cleanup — undo what setup did · Ch 15 Q18

Q68. ⋆ Why does my Effect run twice in development? Because of `<StrictMode>`: in development, React mounts, unmounts and remounts each component once to check your cleanup is symmetrical. It only happens in development. The fix is never to "run it once" with a flag — write a cleanup that undoes the setup (abort the fetch, unsubscribe, clear the timer), so the double run is invisible. → *More:* §5.4 StrictMode's double run (development only) · Ch 15 Q19

Q69. ⋆ How do you avoid infinite loops with `useEffect`? A loop happens when an Effect sets state that changes one of its own dependencies (or has no dependency array and sets state each time). Fixes: don't put an object/array/function created during render in the deps (move it inside the Effect, memoize it, or depend on its primitive fields); use the **updater form** (`setCount(c => c + 1)`) so `count` isn't a dependency; and ask whether you need the Effect at all — derived data should be computed during render. → *More:* §5.5 Dependencies in depth · §5.6 You might not need an Effect

Q70. ⋆ Why does putting an object or function in the dependency array make the Effect run every render? Dependencies are compared by **reference** (`Object.is`). `const options = { roomId }` or `function onMessage() {}` created in the component body is a **new** object every render, so React sees "changed" each time. Create it **inside** the Effect, move it outside the component if it doesn't use props/state, depend on primitives (`options.roomId`), or memoize it with `useMemo` / `useCallback` (the React Compiler does the last automatically). → *More:* §5.5 Dependencies in depth

Q71. Can the Effect callback be `async`? What happens if you return a Promise from `useEffect`? No. React expects the Effect to return nothing or a cleanup **function**; an `async` function returns a Promise, which React warns about and can't use as cleanup. Define an async function inside and call it, with an `ignore` flag or `AbortController` for cleanup:

```
useEffect(() => {
  const ac = new AbortController();
  (async () => {
    const res = await fetch(`/api/users/${id}`, { signal: ac.signal });
    setUser(await res.json());
  })().catch((e) => { if (e.name !== 'AbortError') setError(e); });
  return () => ac.abort();
}, [id]);
```

→ *More:* §5.7 Fetching data in Effects and race conditions

Q72. ⋆ What is a race condition when fetching in an Effect, and how do you prevent it? If `id` changes from 1 to 2 quickly, the request for 1 might answer **after** the request for 2, and the stale result overwrites the correct one. Prevent it in the cleanup: set a local `ignore = true` flag (and skip `setState` if ignored) or `abort()` the previous request with `AbortController`. Libraries like TanStack Query handle this for you. → *More:* §5.7 Fetching data in Effects and race conditions · Ch 15 Q20

Q73. ⋆ What is a stale closure in a Hook, and how do you fix it? A **closure** is a function that remembers the variables from the render where it was created. If an Effect, interval or callback is created once (`[]` deps) but reads `count`, it keeps seeing the **first** render's `count` forever — it's "stale". Fixes: list the value in the dependencies, use the **updater** form (`setCount(c => c + 1)`), keep the latest value in a **ref**, or use `useEffectEvent` for non-reactive logic called from an Effect.

```
useEffect(() => {
  const id = setInterval(() => setCount(c => c + 1), 1000); // not count + 1
  return () => clearInterval(id);
}, []);
```

→ *More:* §5.5 Dependencies in depth · §7.7 Hooks that take callbacks — `useEffectEvent`

Q74. ⋆ "You might not need an Effect" — what are the common unnecessary Effects? (1) Computing derived data (`useEffect(() => setFullName(first + ' ' + last))`) — compute during render. (2) Caching expensive results — `useMemo`. (3) Resetting state when a prop changes — use a `key`. (4) Reacting to a user event (submit, buy, notify) — do it in the **event handler**. (5) Chains of Effects setting state that triggers other Effects — compute the next state in one handler. (6) Notifying the parent of a change — call the parent's callback in the same handler. Effects are for syncing with **external** systems. → *More:* §5.6 You might not need an Effect · Ch 15 Q16

Q75. In which order do Effects run in a parent and its children? Rendering goes top-down (parent body, then child body), but Effects run **bottom-up**: children's Effects run before the parent's, so by the time a parent's Effect runs, its whole subtree is in the DOM and has already run its own Effects. On updates, all relevant **cleanups** run before any new setups. `useLayoutEffect` functions all run before `useEffect` ones. → *More:* §5.3 Cleanup — undo what setup did · Ch 15 Q18

Q76. Can you have multiple `useEffect`s in one component? Should you? Yes, and you should split by **purpose**: one Effect per external system ("sync with the chat connection", "sync with analytics"). Each then has its own correct dependencies and cleanup. One Effect doing unrelated things re-runs all of them whenever any dependency changes. → *More:* §5.2 `useEffect` basics

Q77. What is the difference between `useEffect` with `[]` and `componentDidMount`? Both run after the first mount, but `useEffect` runs **after the browser paints** (componentDidMount ran before paint — closer to `useLayoutEffect`), it captures props/state of that render (closures), and in StrictMode development it

runs setup → cleanup → setup. More importantly, the mental model differs: an Effect describes how to **start and stop syncing**, not a lifecycle moment. → *More:* §5.2 `useEffect` basics · §5.11 `useLayoutEffect` — before the browser paints

Q78. ★ `useEffect` vs `useLayoutEffect`? `useEffect` runs **after** the browser paints — non-blocking, right for most side effects. `useLayoutEffect` runs synchronously **after the DOM is updated but before paint**, so you can **measure** layout (an element's size or position) and set state that adjusts it without a visible flicker (tooltips, popovers, scroll restoration). It blocks painting, so use it only when needed. On the server it does nothing (React warns in old versions); in Server-rendered apps measure on the client only. → *More:* §5.11 `useLayoutEffect` — before the browser paints · Ch 15 Q21

Q79. What is `useInsertionEffect`? A Hook that runs during the commit, **before any layout Effects fire**, intended **only for CSS-in-JS library authors** to inject `<style>` tags before any layout is read. Application code shouldn't need it. → *More:* §8.7 CSS-in-JS

Q80. ★ What is `useRef`, and what are its use cases? `useRef(initial)` returns a mutable object `{ current }` that **persists across renders**; changing `.current` does **not** re-render. Use cases: (1) a reference to a **DOM node** (`<input ref={inputRef}>` to focus, scroll, measure); (2) storing values that don't affect the UI — interval IDs, `AbortController`s, the previous value, "is mounted" flags, a third-party widget instance. Don't read or write `ref.current` during render (except lazy initialisation) — it makes the component impure. → *More:* §5.8 `useRef` for values · §5.9 Refs to DOM nodes

Q81. ★ `useRef` vs `useState`? Both persist across renders. Updating **state** triggers a re-render and the new value is seen on the next render; updating a **ref** is immediate, mutable, and does **not** re-render. Use state for anything shown on screen; use a ref for things only your event handlers/Effects need (timer IDs, DOM nodes, latest values for callbacks). → *More:* §5.8 `useRef` for values · Ch 15 Q22

Q82. ★ How do you focus an input on mount? Use a ref and an Effect (or the `autoFocus` attribute for the simple case):

```
function Search() {
  const inputRef = useRef(null);
  useEffect(() => { inputRef.current.focus(); }, []);
  return <input ref={inputRef} />;
}
```

→ *More:* §5.9 Refs to DOM nodes

Q83. ★ How do you get the previous value of a prop or state (`usePrevious`)? The classic answer stores the value in a ref after each commit:

```
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; }, [value]);
  return ref.current; // value from the previous render
}
```

Note it reads a ref during render, which React's docs discourage. The React-recommended alternative keeps the previous value in **state** and updates it during render when it differs (`if (value !== prev) { setPrev(value); … }`). → *More:* §5.8 `useRef` for values

Q84. ★ How do you pass a ref to your own component in React 19? What was `forwardRef`? In React 19, `ref` is an **ordinary prop** for function components: `function Input({ ref, ...props }) { return <input ref={ref} {...props} /> }`. Before 19 you had to wrap the component in `forwardRef((props, ref) => …)`; it still works but is planned for deprecation. Class components still don't receive `ref` as a prop. → *More:* §5.10 Passing refs to your own components · Ch 15 Q23

Q85. What is `useImperativeHandle`, and when should you use it? It lets a component **customise what a parent's ref receives** — instead of the DOM node, expose a small API: `useImperativeHandle(ref, () => ({ focus() {…}, scrollToTop() {…} }), [])`. Use it rarely, for imperative actions that can't be expressed as props (focus, play/pause a video, open a native dialog). In React 19 it works with `ref` taken as a prop — no `forwardRef` needed. → *More:* §5.10 Passing refs to your own components

Q86. What are callback refs, and what changed about them in React 19? Instead of a ref object you can pass a **function**: `<div ref={node => …}>`. React calls it with the DOM node when attached. React 19 lets

the callback **return a cleanup function**, called when the node is detached (instead of the old behaviour of calling the callback with `null`). Useful for measuring nodes in a list or attaching observers per element. → *More:* §5.9 Refs to DOM nodes

Q87. Can you read a ref during rendering? You shouldn't. `ref.current` is not tracked by React, so a render that reads it may show stale data and breaks purity (and the React Compiler will skip such components). DOM refs are also `null` during the first render because the node doesn't exist yet. Read and write refs in **event handlers and Effects**. → *More:* §5.8 `useRef` for values

---

6. Other Hooks, Rules of Hooks and custom Hooks

Q88. ⋆ What are Hooks, and why were they introduced? Hooks are functions (`useState`, `useEffect`, `useContext` …) that let function components use state and other React features. Introduced in React 16.8 (2019) because classes made logic reuse hard (HOCs and render props produced "wrapper hell"), spread related logic across lifecycle methods, and `this` confused people. Hooks let you group logic by concern and share it through **custom Hooks**. → *More:* §7.1 What a Hook really is

Q89. ⋆ What are the Rules of Hooks, and why do they exist? (1) Call Hooks only at the **top level** of a component or custom Hook — not in conditions, loops, nested functions, after an early `return`, or in `try/catch`. (2) Call them only from **React function components or custom Hooks**, not from ordinary functions or classes. React identifies each Hook's state by its **call order** in the component; a Hook called conditionally shifts the order and every Hook after it reads the wrong state. Enforced by `eslint-plugin-react-hooks`. (`use()` is the exception: it may be called conditionally.) → *More:* §7.2 The Rules of Hooks · Ch 15 Q25

Q90. What happens if you call `useState` conditionally? On a render where the condition flips, the number or order of Hook calls changes. React matches state by position, so later Hooks get the wrong values, and React throws "Rendered more/fewer hooks than during the previous render". Move the condition **inside** the Hook usage (or into an Effect body) instead. → *More:* §7.2 The Rules of Hooks

Q91. How does `useState` work internally (high level)? Each component instance has a **fiber** that stores a linked list of its Hooks in call order. On the first render `useState` creates a slot with the initial value and an update queue; the setter adds an update to that queue and schedules a render. On the next render, the n-th Hook call reads the n-th slot, applies the queued updates in order, and returns the result. That's why call order must be stable. → *More:* §7.1 What a Hook really is

Q92. ⋆ What is a custom Hook? Do two components using the same Hook share state? A **custom Hook** is a function whose name starts with `use` and which calls other Hooks, used to share **stateful logic** (not state) between components: `useFetch`, `useDebounce`, `useLocalStorage`, `useOnlineStatus`. Each call gets its **own independent state** — two components calling `useCounter()` have separate counters. To share the data itself, lift state up, use Context or a store. → *More:* §7.3 Custom Hooks — reusing stateful logic · Ch 15 Q26

Q93. Why must a custom Hook's name start with `use`? It's how React's lint rules and the React Compiler know the function may call Hooks, so they can check the Rules of Hooks inside and at the call site. A function that doesn't call any Hook shouldn't be named `use…` — make it a plain helper. → *More:* §7.3 Custom Hooks — reusing stateful logic

Q94. ⋆ What is `useMemo`, and when should you use it? `useMemo(() => compute(a, b), [a, b])` **caches the result** of a calculation between renders and recomputes only when a dependency changes. Use it for (1) genuinely **expensive** calculations (sorting/filtering thousands of items — measure first), and (2) keeping a **stable object/array reference** passed to a `memo` child or used as an Effect dependency. Don't wrap cheap computations — it adds overhead and noise. It's a performance hint, not a guarantee (React may drop the cache). → *More:* §10.6 `useMemo` for expensive calculations · §10.5 `useCallback` and `useMemo` for stable props

Q95. ⋆ What is `useCallback`, and why is it used? `useCallback(fn, deps)` returns the **same function object** between renders until a dependency changes. Functions are re-created on every render, so without it a function prop breaks `memo` on a child, or retriggers an Effect that depends on it. It doesn't make the function faster. `useCallback(fn, deps)` equals `useMemo(() => fn, deps)`. → *More:* §10.5 `useCallback` and `useMemo` for stable props

Q96. ⋆ `useMemo` vs `useCallback` — what's the difference? `useMemo` caches the **value returned** by calling a function; `useCallback` caches the **function itself**. Use `useMemo` for computed data and objects, `useCallback` for event handlers passed to memoized children or used in Effect deps. Both only matter when someone compares references (`memo`, dependency arrays). → *More:* §10.5 `useCallback` and `useMemo` for stable props · Ch 15 Q34

Q97. Does `useMemo` prevent a child from re-rendering? Not on its own. A child re-renders when its parent re-renders, regardless of props — unless the child is wrapped in `memo`. `useMemo`/`useCallback` only keep props referentially stable so `memo`'s shallow comparison can succeed. (Exception: memoizing the JSX element itself, `useMemo(() => <Child/>, [...])`, does let React skip it.) → *More:* §10.4 `memo` — skipping a child's re-render · §10.5 `useCallback` and `useMemo` for stable props

Q98. ⋆ What is `useReducer`, and when would you choose it over `useState`? `const [state, dispatch] = useReducer(reducer, initialState)` keeps state that you change by **dispatching actions** (`dispatch({ type: 'added', text })`) to a pure **reducer** function `(state, action) => newState`. Choose it when several values change together, the next state depends on complex rules, many event handlers update the same state, or you want update logic testable outside the component. `useState` is simpler for independent values. → *More:* §6.2 `useReducer` — consolidate update logic · Ch 15 Q27

Q99. What rules should a reducer follow? It must be **pure**: no side effects (no fetching, no timers, no logging to servers), no mutation of `state` or `action` — return a new object — and the same input must give the same output (no `Math.random()`/`Date.now()` inside; pass such values in the action). Handle unknown actions by throwing or returning state. React may call reducers twice in StrictMode to check this. → *More:* §6.2 `useReducer` — consolidate update logic

Q100. Is `dispatch` asynchronous? Can you dispatch several actions in a row? Like a state setter, `dispatch` **queues** the action and schedules a re-render; the `state` variable in the current render doesn't change. Several dispatches in one event are batched into one render, and each action is applied to the result of the previous one. `dispatch` has a stable identity, so it's safe to pass down or omit from Effect deps. → *More:* §6.2 `useReducer` — consolidate update logic

Q101. ⋆ What is Context, and how does `useContext` work? Context passes a value deeply without props. `const ThemeCtx = createContext('light')` creates it; `<ThemeCtx value="dark">…</ThemeCtx>` provides it (React 19 — earlier `<ThemeCtx.Provider value>`); `useContext(ThemeCtx)` (or `use(ThemeCtx)`) reads the value from the **nearest provider above**. When the provided value changes, every component that reads it re-renders. Typical uses: theme, current user, locale, a store or service instance. → *More:* §6.3 Context — pass data deeply without props

Q102. What does a component get from `useContext` when there's no provider? What's the default value for? It gets the **default value** passed to `createContext(default)`. The default is useful for tests, for optional contexts, and as documentation of the shape. Many teams use `createContext(null)` and a custom Hook that throws a clear error if no provider is present (`if (!ctx) throw new Error('useCart must be used inside <CartProvider>')`). → *More:* §6.3 Context — pass data deeply without props · §11.7 Typing custom Hooks and Context

Q103. ⋆ What is the common pitfall when passing an object as a Context value? `<Ctx value={{ user, setUser }}>` creates a **new object every render** of the provider, so every consumer re-renders even when `user` didn't change. Fix: memoize the value with `useMemo`, **split** contexts (state vs dispatch/actions, or rarely vs often changing data), and keep providers low in the tree. Context has no selector — a consumer re-renders on any change of the value, and `memo` on the consumer doesn't stop it. → *More:* §6.4 Context performance caveats · Ch 15 Q29

Q104. ⋆ Is Context a state-management tool? Context vs Redux? Context is a **transport** (dependency injection): it delivers a value; `useState`/`useReducer` in the provider holds the state. It's fine for low-frequency global values. Redux (or Zustand) is a **store** with selective subscriptions (components re-render only when their selected slice changes), middleware, DevTools time-travel and conventions for large teams. Use Context for theme/user/locale; a store for frequently updated, widely shared client state. → *More:* §6.3 Context — pass data deeply without props · §6.7 Redux Toolkit · Ch 15 Q28

Q105. What is `useId`? `useId()` generates a **unique, stable ID** per component instance that is the **same on the server and the client**, so it doesn't cause hydration mismatches. Use it to link labels and ARIA attributes: `<label htmlFor={id}>` + `<input id={id}>`, `aria-describedby`. Don't use it for list keys. → *More:* §7.8 `useId` — IDs for accessibility

Q106. ⋆ What are `useTransition` and `useDeferredValue`, and how do they differ? Both mark work as **non-urgent** so typing and clicks stay responsive. `useTransition` returns `[isPending, startTransition]`; you wrap the **state update** yourself (`startTransition(() => setTab(next))`) — use it when you own the setter. `useDeferredValue(value)` gives a **lagging copy** of a value you receive (often a prop); React renders the urgent UI with the old copy first, then re-renders with the new one in the background. Neither debounces network requests. → *More:* §7.10 `useTransition` — non-urgent updates · §7.11 `useDeferredValue` — let part of the UI lag behind · Ch 15 Q41

Q107. What is `useSyncExternalStore`? A Hook for subscribing to data that lives **outside React** (a store, browser APIs like `navigator.onLine` or `matchMedia`, a third-party library) in a way that's safe with concurrent rendering (no "tearing", where different components show different versions). Signature: `useSyncExternalStore(subscribe, getSnapshot, getServerSnapshot?)`. Store libraries (Redux, Zustand) use it internally. → *More:* §7.9 `useSyncExternalStore` — subscribing to outside data · Ch 15 Q32

Q108. ⋆ What is `useEffectEvent` (React 19.2)? It creates an **Effect Event**: a function you call from inside an Effect that always sees the **latest** props and state but is **not reactive** — it doesn't need to be (and must not be) in the dependency array. Use it for the "read but don't re-sync on" part of an Effect: e.g. reconnect only when `roomId` changes, but show a notification using the current `theme`. Call it only from inside Effects (or other Effect Events) — never during render, from an `onClick`, or by passing it to other components or Hooks. Its identity deliberately **changes every render**, so it is not a stable callback and must never appear in a dependency array. → *More:* §7.7 Hooks that take callbacks — `useEffectEvent` · Ch 15 Q24

Q109. What is `useDebugValue`? A Hook for custom-Hook authors: `useDebugValue(isOnline ? 'Online' : 'Offline')` shows a label next to the Hook in React DevTools. It has no effect on behaviour; use it in shared library Hooks. → *More:* §16.3 Every Hook in one table

Q110. Do Hooks replace HOCs and render props? Mostly. Custom Hooks share stateful logic without adding wrapper components or changing the tree. HOCs and render props are still used when you need to wrap rendering (e.g. a boundary, a layout, a library API built on them), and error boundaries are still classes. In new code, prefer a custom Hook first. → *More:* §14.7 Choosing a reuse pattern · Ch 15 Q50

Q111. What are best practices for Hooks? Follow the Rules of Hooks with the lint plugin on; keep Effects for external syncing only; list every dependency honestly (fix the code, not the array); one Effect per concern with a cleanup; prefer derived values and event handlers to state + Effects; extract custom Hooks for repeated logic and name them by purpose (`useOnlineStatus`, not `useMount`); avoid premature `useMemo`/`useCallback` (and let the React Compiler handle memoization where it's enabled). → *More:* §7.2 The Rules of Hooks · §5.6 You might not need an Effect

---

7. React 19 features

Q112. ⋆ What is an Action in React 19? An **Action** is a function — usually **async** — that you run inside a **transition**, typically to submit or mutate data. React tracks it for you: `isPending` is true while it runs, errors go to the nearest error boundary, and optimistic updates are rolled back when it finishes. One caveat: inside a hand-written `startTransition(async () => …)`, state updates made **after an** `await` are not part of the transition — wrap them in another `startTransition` (a known limitation until JavaScript gets async context). You get Actions by passing a function to `startTransition`, to `<form action>`, `<button formAction>`, or `useActionState`. → *More:* §4.4 Form Actions — `<form action={fn}>` · Ch 15 Q14

Q113. ⋆ How do forms work with `<form action={fn}>`? Pass a function to the `action` prop; on submit React calls it with a `FormData` object (the form's fields by `name`), inside a transition, and **resets** uncontrolled fields after a successful action. No `e.preventDefault()`, no `onSubmit`, no controlled inputs needed:

```
async function save(formData) { await api.addTodo(formData.get('title')); }
<form action={save}><input name="title" /><button>Add</button></form>
```

If the function is a **Server Function**, the form even works before JavaScript loads (progressive enhancement). → *More:* §4.4 Form Actions — `<form action={fn}>`

Q114. ⋆ What does `useActionState` do? `const [state, formAction, isPending] = useActionState(action, initialState)` wraps an action so its **return value becomes state**. Your action receives `(previousState, formData)`; whatever it returns (an error message, the saved item, field errors) becomes the new `state`. Pass `formAction` to `<form action>`. It replaced the canary-only `useFormState` from `react-dom`. Ideal for showing validation/server errors after a submit.

```
const [error, submit, pending] = useActionState(async (prev, fd) => {
  const res = await signup(fd.get('email'));
  return res.ok ? null : res.message;
}, null);
```

→ *More:* §4.5 `useActionState` — the result of the last submission

Q115. ⋆ What does `useFormStatus` do, and what's its main gotcha? `const { pending, data, method, action } = useFormStatus()` (from `react-dom`) tells a component the status of the `<form>` **it is rendered inside** — perfect for a reusable `<SubmitButton>` that disables itself while submitting. Gotcha: it only reads a **parent** form, so it returns `pending: false` if called in the same component that renders the `<form>`; put it in a child component. → *More:* §4.6 `useFormStatus` — status of the parent form

Q116. ⋆ What does `useOptimistic` do? `const [optimisticItems, addOptimistic] = useOptimistic(items, (current, newItem) => [...current, newItem])` shows a **temporary** version of the state while an Action is running — e.g. the new comment appears instantly with a "sending…" style. When the Action ends, React drops the optimistic value and shows the real state again (so on failure it rolls back automatically). Call `addOptimistic` inside an Action/transition. → *More:* §4.7 `useOptimistic` — instant feedback, automatic rollback

Q117. ⋆ What is the `use()` API? How is it different from other Hooks? `use(resource)` reads a **promise** or a **context**. With a promise, the component **suspends** (the nearest `<Suspense>` fallback shows) until it resolves, then gets the value; a rejection goes to the nearest error boundary. Unlike Hooks, `use` **can be called conditionally** and in loops (but not inside `try/catch`). The promise must be **stable** — created in a Server Component, a loader or a cache — not created fresh during a Client Component's render: a new promise every render suspends again each time (React warns about an "uncached promise"). → *More:* §9.11 Suspense and `use()` · Ch 15 Q40

Q118. `use(Context)` vs `useContext(Context)`? They return the same value. `use` is more flexible because it can be called **after an early return or inside an `if`**: `if (!show) return null; const theme = use(ThemeCtx);`. `useContext` must follow the Rules of Hooks. → *More:* §6.3 Context — pass data deeply without props · §9.11 Suspense and `use()`

Q119. ⋆ What are React Server Components (RSC)? Server Components run **only on the server** (at build time or per request) and send their rendered result — not their code — to the browser. They can be `async`, read databases and files directly, keep secrets on the server, and add **zero JavaScript** to the client bundle. They can't use state, Effects, event handlers or browser APIs. **Client Components** (marked by `'use client'` at the top of a module) are the interactive parts and are still server-rendered to HTML then hydrated. They need a framework/bundler that supports them (Next.js App Router, React Router's RSC support). → *More:* §13.7 React Server Components · Ch 15 Q45

Q120. ⋆ When do you choose a Server Component vs a Client Component? Server Component (the default in Next.js App Router): fetching data, accessing backend resources or secrets, rendering large static content, heavy dependencies (markdown parsers, date libraries) you don't want to ship. Client Component: anything interactive — state, Effects, event handlers, browser APIs (`localStorage`, `window`), custom Hooks using these, and most third-party UI libraries. Push `'use client'` down to the **leaves** (a `LikeButton`, not the whole page). → *More:* §13.7 React Server Components · §13.8 Directives: `'use client'` and `'use server'`

Q121. ⋆ What are Server Functions (Server Actions)? What does `'use server'` mean? A **Server Function** is an async function that runs on the server but can be **called from the client** — React sends a request under the hood. Mark it with `'use server'` (at the top of the function body or of a module). Used as a form `action` or called from an event handler/transition, it performs mutations (save to DB, then revalidate). `'use server'` does **not** mark Server Components — those need no directive. Treat every Server Function as a public API endpoint: validate input and check authorisation. → *More:* §4.8 Server

Functions and progressive enhancement (overview) · §13.8 Directives: `'use client'` and `'use server'` · Ch 15 Q46

Q122. Can a Server Component import a Client Component, and vice versa? A Server Component can import and render a Client Component, passing **serialisable** props (plain data, JSX, promises, Server Functions — not ordinary functions or class instances). A Client Component **cannot import** a Server Component (the import would become client code), but it can **receive one as `children` or another prop** that was rendered by a Server Component parent. → *More:* §13.7 React Server Components

Q123. ⋆ What is the React Compiler? A **build-time** tool (a Babel plugin, stable since v1.0 in October 2025) that analyses components and Hooks and **automatically memoizes** values, callbacks and JSX — like perfectly placed `useMemo`, `useCallback` and `memo`. It requires code that follows the **Rules of React** (pure render, no mutation of props/state, Rules of Hooks) and skips components it can't prove safe. With it, manual memoization is mostly unnecessary; it doesn't fix slow algorithms, large DOMs or big bundles. Opt a component out with `'use no memo'`. → *More:* §10.7 React Compiler · Ch 15 Q35

Q124. What other React 19 changes might an interviewer ask about? `<Context value>` as provider; **ref cleanup** functions; `useDeferredValue(value, initialValue)`; rendering `<title>`, `<meta>`, `<link>` anywhere (hoisted to `<head>`); stylesheets with `precedence`; `async` scripts deduplicated; **resource APIs** `preload`, `preinit`, `prefetchDNS`, `preconnect` from `react-dom`; clearer **hydration-mismatch** errors with a diff; errors no longer re-thrown, with new root options `onCaughtError` / `onUncaughtError` / `onRecoverableError`; full support for **custom elements**. 19.2 added `<Activity>` (hide UI while keeping its state), `useEffectEvent`, `cacheSignal` and Performance Tracks in Chrome DevTools. → *More:* §5.10 Passing refs to your own components · §14.9 Error boundaries · §13.5 Hydration mismatches

Q125. What was removed or deprecated in React 19 (migration questions)? Removed: `ReactDOM.render` / `hydrate`, `unmountComponentAtNode`, `findDOMNode`, `propTypes` checks, `defaultProps` on function components, legacy context (`contextTypes`), string refs, module-pattern factories, `react-test-renderer/shallow`; `act` moved to `react`. Deprecated: `forwardRef` (planned), `element.ref` (ref is a prop now), `react-test-renderer`. TypeScript: `useRef` requires an argument, refs are mutable, `JSX` namespace moved to `React.JSX`. A codemod (`npx codemod react/19/migration-recipe`) automates much of it. → *More:* §11.1 Setup and how type-checking fits in · §16.18 Common errors and fixes

Q126. What is `<Activity>`? A React 19.2 component: `<Activity mode={visible ? 'visible' : 'hidden'}>`. Hidden content keeps its **state** (and DOM), but React hides it with `display: none`, cleans up its Effects (they run again when it becomes visible), and re-renders it only at low priority. Use it for tabs or panels you want to switch back to instantly, or to pre-render the likely next screen. → *More:* §5.4 StrictMode's double run (development only)

Q127. What are transitions and why do Actions use them? A **transition** marks a state update as non-urgent (`startTransition(() => …)`): React can interrupt it, keeps showing the current UI instead of a Suspense fallback, and exposes `isPending`. Since React 19 `startTransition` accepts **async** functions, which is what an Action is — so pending state, error handling and optimistic updates are all tied to one transition. → *More:* §7.10 `useTransition` — non-urgent updates · §4.4 Form Actions — `<form action={fn}>`

---

8. State management: Context, Redux, Zustand and server state

Q128. ⋆ How do you decide between local state, Context and an external state manager? Start **local** (`useState` in the component, lifted only as high as needed). Use the **URL** for state that should survive refresh or be shareable (filters, page, tab). Use a **server-state library** (TanStack Query, RTK Query, framework loaders) for data from APIs. Use **Context** for rarely changing global values (theme, user, locale). Reach for a **store** (Zustand, Redux Toolkit) when lots of client state is shared and updated often, or you need devtools/middleware and team conventions. → *More:* §6.10 Choosing — comparison and decision flow · Ch 15 Q30

Q129. ⋆ What is Redux, and what are its core principles? **Redux** is a predictable state container: (1) a **single source of truth** — the whole app state in one store; (2) **state is read-only** — the only way to change it is to dispatch an **action** (a plain object `{ type, payload }`); (3) changes are made by **pure reducers** `(state, action) => newState`. This makes every change traceable (Redux DevTools, time-travel

debugging) and testable. Today you write Redux with **Redux Toolkit** and the `react-redux` Hooks `useSelector` and `useDispatch`. → *More:* §6.7 Redux Toolkit

Q130. ⋆ Why is Redux Toolkit (RTK) recommended over "plain" Redux? RTK is the official way to write Redux. It removes boilerplate: `configureStore` (sensible defaults, DevTools, thunk middleware, mutation checks), `createSlice` (action creators and reducer in one place, with **Immer** so you can write "mutating" updates), `createAsyncThunk` for async logic, `createSelector` for memoized selectors, and **RTK Query** for data fetching and caching. Hand-written action types, switch statements and spread-heavy reducers are no longer needed. → *More:* §6.7 Redux Toolkit

Q131. What happens when you dispatch an action in Redux? `dispatch(action)` passes the action through the **middleware** chain (thunks, logging …), then the store calls the root **reducer** with the current state and the action; the reducer returns the new state. The store notifies subscribers; each component's `useSelector` re-runs its selector and the component re-renders **only if the selected value changed** (compared by reference). → *More:* §6.7 Redux Toolkit

Q132. What is Redux middleware? What are thunks? Middleware wraps `dispatch` to add behaviour between dispatching and the reducer: logging, crash reporting, async logic. Its signature is `store => next => action => { … next(action) … }`. A **thunk** is a function you dispatch instead of an object: `dispatch(async (dispatch, getState) => { … })`, letting you run async work and dispatch real actions later. `createAsyncThunk` generates `pending` / `fulfilled` / `rejected` actions for you. Redux-Saga (generators) and Redux-Observable (RxJS) are heavier alternatives. → *More:* §6.7 Redux Toolkit

Q133. What are selectors and `createSelector`? How do you prevent unnecessary re-renders with Redux? A **selector** is a function that reads part of the state: `state => state.cart.items`. `useSelector` re-renders when the selector's result changes **by reference**, so selectors that build new arrays/objects (`state.items.filter(...)`) re-render every time. Use `createSelector` (Reselect) to **memoize** derived data — it recomputes only when its inputs change. Select the smallest piece you need, and use `shallowEqual` when returning objects. → *More:* §6.7 Redux Toolkit

Q134. What is normalised state, and why does it help? Normalised state stores each entity **once**, in a lookup by ID — `{ ids: [1, 2], entities: { 1: {...}, 2: {...} } }` — and refers to it by ID elsewhere, like database tables. Updates touch one place, there are no duplicate copies to get out of sync, and lookups are O(1). RTK's `createEntityAdapter` generates this structure and its reducers. → *More:* §6.7 Redux Toolkit

Q135. Should all state go into Redux? No. Keep **local UI state** (an open dropdown, a form's draft input) in components, **server data** in a server-state cache (RTK Query / TanStack Query), and URL state in the URL. Put in Redux only state that many unrelated parts of the app read and update, or that needs time-travel/debugging. Over-centralising makes every keystroke a global action. → *More:* §6.1 Kinds of state — and where each belongs · §6.6 When to reach for a library

Q136. ⋆ What is Zustand? How does it compare with Redux and Context? Zustand is a tiny store library: `const useCart = create((set) => ({ items: [], add: (i) => set((s) => ({ items: [...s.items, i] })) }))`, then `useCart((s) => s.items)` in any component — **no provider**, no actions/reducers ceremony. Components re-render only when their **selected** value changes (unlike Context). Compared with Redux it has less structure and tooling but far less code. Gotcha (v5): a selector returning a new object each time causes an infinite loop — wrap it in `useShallow`. → *More:* §6.8 Zustand

Q137. ⋆ What is the difference between client state and server state? What is TanStack Query (React Query)? Client state is owned by the browser (UI toggles, form drafts) — synchronous and always up to date. **Server state** lives on a server: you only have a **cached copy** that can go stale, needs loading/error states, deduplication, refetching and invalidation. **TanStack Query** manages server state: `useQuery({ queryKey: ['todos', page], queryFn })` gives `data`, `isPending`, `error`, caching by key, background refetch, retries; `useMutation` + `invalidateQueries` refresh data after writes. → *More:* §6.9 TanStack Query — server state · §9.8 TanStack Query basics · Ch 15 Q31

Q138. What is Flux? Flux is the architecture Facebook proposed alongside React: data flows one way — **actions** go through a **dispatcher** to **stores**, which update and notify **views**, which create new actions. Redux simplified it to a single store and pure reducers. It's mainly asked as history/background. → *More:* §6.7 Redux Toolkit

Q139. MobX, Jotai, Recoil — what are they? MobX: observable objects with automatic dependency tracking — mutate state directly and observing components update (OOP/reactive style). **Jotai**: **atoms** — small

independent pieces of state composed bottom-up, with fine-grained re-renders. **Recoil**: Meta's atom library, now **archived** (unmaintained) — don't choose it for new projects. The trade-off across all: less boilerplate vs. Redux's explicitness and tooling. → *More:* §6.10 Choosing — comparison and decision flow

Q140. How do you keep a user logged in across page refreshes with Context? React state is lost on reload, so persist the **source of truth** outside React: an **HTTP-only cookie** session (safest — JavaScript can't read it, so XSS can't steal it), and on app start call `/api/me` to restore the user into Context (show a loading state meanwhile). Avoid storing long-lived tokens in `localStorage`, because any XSS can read it. Server-rendering frameworks can read the cookie on the server and render the logged-in page directly. → *More:* §6.3 Context — pass data deeply without props · §9.7 Middleware, lazy routes and framework mode

---

9. Forms, routing and data fetching

Q141. ⋆ Controlled vs uncontrolled inputs — what's the difference and when to use each? Controlled: React state holds the value — `<input value={name} onChange={e => setName(e.target.value)} />` — so you can validate, format or disable on every keystroke. **Uncontrolled**: the DOM holds the value; you read it on submit via `FormData` or a ref, with `defaultValue` for the initial value. React 19 form Actions make uncontrolled forms the simpler default; use controlled when the UI must react to each change (live validation, dependent fields). → *More:* §4.2 Controlled vs uncontrolled inputs · Ch 15 Q13

Q142. ⋆ Why do I get "A component is changing an uncontrolled input to be controlled"? The `value` prop went from `undefined` (uncontrolled) to a string (controlled), usually because state started as `undefined` or a field was missing on an object. Initialise with an empty string (`useState('')`, `value={user.name ?? ''}`) and keep the input controlled for its whole life. → *More:* §4.2 Controlled vs uncontrolled inputs · §16.18 Common errors and fixes

Q143. How do you handle multiple inputs with one change handler? Give each input a `name` and update by key:

```
const [form, setForm] = useState({ email: '', password: '' });
const onChange = (e) => setForm({ ...form, [e.target.name]: e.target.value });
<input name="email" value={form.email} onChange={onChange} />
```

For checkboxes use `e.target.checked`. With form Actions you may not need state at all — read `formData.get('email')`. → *More:* §4.2 Controlled vs uncontrolled inputs

Q144. ⋆ How do you validate forms in React? Layers: (1) **native HTML** attributes (`required`, `type="email"`, `minLength`, `pattern`) — free and accessible; (2) **custom checks** on submit (and on blur for touched fields), storing an `errors` object and rendering messages linked with `aria-describedby` and `aria-invalid`; (3) a **schema** (Zod) shared by client and server; (4) the **server always re-validates**. Libraries: React Hook Form + Zod resolver for large forms. Show errors after the user leaves a field or submits, not on the first keystroke. → *More:* §4.3 Validation · §4.9 Form libraries — React Hook Form and Zod (overview)

Q145. What are React Hook Form and Formik? Why use a form library? They manage values, touched/dirty flags, validation and submission for big forms. **React Hook Form** uses uncontrolled inputs registered with refs (`register('email')`), so typing doesn't re-render the whole form — fast and small, with Zod/Yup resolvers. **Formik** is controlled and older, re-renders more, and is less actively maintained. For simple forms, React 19 Actions + native validation are often enough. → *More:* §4.9 Form libraries — React Hook Form and Zod (overview)

Q146. How do you handle file uploads in React? Use an uncontrolled `<input type="file">` (its value can't be set by script). Read `e.target.files` or `formData.get('file')`, and send with `fetch` and a `FormData` body (don't set `Content-Type` manually — the browser adds the multipart boundary). For progress, use `XMLHttpRequest`'s `upload.onprogress` (fetch has no upload progress); preview images with `URL.createObjectURL(file)` and revoke it in cleanup. → *More:* §4.1 How a form submission works

Q147. ⋆ What is client-side routing, and how does React Router work? In a **single-page app (SPA)** the browser loads one HTML page; a router reads the URL and renders the matching components, and `<Link>` changes the URL with the **History API** (`pushState`) instead of loading a new page. React Router v8 (one

`react-router` package) has three modes: **declarative** (`<BrowserRouter>`, `<Routes>`, `<Route>`), **data** (`createBrowserRouter` + `<RouterProvider>` with loaders/actions) and **framework** (route modules, SSR). → *More:* §9.1 Client-side routing · §9.2 React Router v8 and its three modes

Q148. How do you read URL params and query strings, and navigate programmatically? `const { id } = useParams()` for `/users/:id`; `const [searchParams, setSearchParams] = useSearchParams()` for `?q=…`; `const navigate = useNavigate(); navigate('/home', { replace: true })` after a successful action; `<Navigate to="/login" replace />` to redirect during render. `replace` swaps the current history entry instead of adding one (so Back doesn't return to the form). → *More:* §9.3 Routes, links and URL parameters (declarative mode)

Q149. How do you implement nested routes, layouts and a 404 page? Nest `<Route>`s (or route objects with `children`); the parent renders shared UI (header, sidebar) and an `<Outlet />` where the matching child goes. An `index` route renders at the parent's exact path. A catch-all `path="*"` route renders the Not Found page. In data mode, `errorElement` / `ErrorBoundary` handles loader errors and unknown routes. → *More:* §9.4 Nested routes and layouts

Q150. ⋆ How do you implement protected (private) routes? Declarative mode: a wrapper that checks auth and either renders `<Outlet/>`/children or `<Navigate to="/login" replace state={{ from: location }} />`. Data/framework mode: **middleware** or a loader that throws `redirect('/login?next=…')` before the page loads (no flash of protected content). Remember client-side guards are **UX only** — the API must still reject unauthorised requests.

```
function RequireAuth() {
  const { user } = useAuth(); const location = useLocation();
  return user ? <Outlet /> : <Navigate to="/login" replace state={{ from: location }} />;
}
```

→ *More:* §9.7 Middleware, lazy routes and framework mode

Q151. What are loaders and actions in React Router? A **loader** is a function attached to a route that fetches its data **before** the route renders (in parallel for nested routes), read with `useLoaderData()` — no fetch-in-Effect waterfalls. An **action** handles a route's mutations (`<Form method="post">`), after which loaders re-run automatically so the page shows fresh data. `useNavigation()` exposes pending state; `useFetcher()` does mutations without navigating. → *More:* §9.5 Loaders — data before render (data mode) · §9.6 Actions, `<Form>` and fetchers

Q152. ⋆ How do you fetch data in React? Compare the options. (1) **`useEffect` + `fetch`** — fine for learning, but you must handle loading, errors, races, caching and no SSR yourself. (2) A **data library** — TanStack Query, SWR, RTK Query — caching, dedupe, retries, refetch. (3) **Router loaders** — data starts loading with navigation, in parallel. (4) **Server Components** / framework data APIs — fetch on the server, no client JS. (5) **`use()` + Suspense** with a cached promise. In production prefer 2–4. → *More:* §5.7 Fetching data in Effects and race conditions · §9.8 TanStack Query basics · §9.5 Loaders — data before render (data mode)

Q153. ⋆ What are common pitfalls when fetching data in React? Race conditions (stale response wins) — no cleanup; **waterfalls** (child fetches only after parent's data arrives) — hoist or parallelise with loaders; no loading/error/empty states; fetching in an Effect with an object dependency — infinite loop; double requests in StrictMode dev with no abort; no caching so every mount refetches; setting state after unmount; and leaking secrets by calling third-party APIs with keys from the browser. → *More:* §5.7 Fetching data in Effects and race conditions · §9.12 Loading and error states that users like

Q154. What is Suspense, and how is it used for data fetching? `<Suspense fallback={<Spinner/>}>` shows the fallback while any component inside **suspends** — waiting for lazy code (`lazy`), a promise read with `use()`, or a Suspense-enabled library (TanStack Query's `useSuspenseQuery`, framework loaders). It lets you declare loading states by **placement** instead of `if (loading)` in every component, and enables streaming SSR. Effects or `useEffect` fetches don't trigger Suspense. → *More:* §9.11 Suspense and `use()` · Ch 15 Q39

Q155. How do you cancel a request or debounce API calls in a search box? Debounce the **value** (`useDebounce(query, 300)`) so the fetch runs only after the user pauses, and **abort** the previous request in the Effect's cleanup with `AbortController`. Also ignore empty queries and cache results per query.

TanStack Query does the cancel/caching part when you put the query in the key. → *More:* §7.5 `useDebounce` — wait until the user stops · Ch 15 Q20

Q156. What are optimistic updates? How do you roll back on error? Update the UI **before** the server confirms (a like count goes up instantly), then reconcile with the real response. In React 19 use `useOptimistic` inside an Action — rollback is automatic when the Action ends. With TanStack Query: in `onMutate` snapshot and update the cache, in `onError` restore the snapshot, in `onSettled` invalidate. Use only when success is very likely. → *More:* §4.7 `useOptimistic` — instant feedback, automatic rollback · §9.10 Mutations, invalidation and optimistic updates

Q157. How do you handle authentication tokens and API calls (interceptors, refresh)? Centralise requests in one API module (a `fetch` wrapper or axios instance) that attaches credentials and handles `401`: try a **refresh** once, retry the original request, otherwise log out. Prefer **HTTP-only, Secure, SameSite cookies** over tokens in `localStorage`. Keep auth state in Context/store and guard routes. → *More:* §9.7 Middleware, lazy routes and framework mode

---

10. Performance

Q158. ⋆ How do you optimise a React application's performance? Measure first (React DevTools Profiler, Performance panel, Lighthouse/Web Vitals). Then: fix unnecessary re-renders (move state down, lift content up as `children`, split contexts, `memo` + stable props — or enable the **React Compiler**); make expensive work cheaper (`useMemo`, better algorithms, `useDeferredValue`/transitions); render less DOM (**virtualise** long lists, paginate); ship less JS (**code splitting** with `lazy`, smaller dependencies, Server Components); optimise loading (SSR/streaming, images with sizes and lazy loading, caching with a data library). → *More:* §10.11 A performance checklist · Ch 15 Q36

Q159. ⋆ What is `React.memo`, and when doesn't it help? `memo(Component)` makes React **skip re-rendering** the component when its props are **shallowly equal** (each prop compared with `Object.is`) to last time. It doesn't help when a prop is a new object/array/function every render (fix with `useMemo`/`useCallback`), when `children` JSX is passed (new elements each render), when the component's own state or a context it reads changes, or when the component is cheap anyway. A custom comparison can be passed as the second argument. → *More:* §10.4 `memo` — skipping a child's re-render · Ch 15 Q33

Q160. How do you prevent unnecessary re-renders without `memo`? Two composition tricks: **move state down** — put the state into a small component so only it re-renders (e.g. an input's state in `<SearchBox>` instead of the page); and **lift content up** — pass expensive subtrees as `children` to the component that owns the changing state, so they are created by the parent and keep the same element reference. Also keep context values stable and avoid creating components inside components. → *More:* §10.2 When components re-render · §10.10 Common performance bugs

Q161. ⋆ What is code splitting? How do `lazy` and `Suspense` work? Code splitting breaks the bundle into chunks loaded on demand, so the first page downloads less JavaScript. `const Settings = lazy(() => import('./Settings'))` creates a component whose code is fetched (via dynamic `import()`) the first time it renders; wrap it in `<Suspense fallback={…}>` to show something meanwhile. Split by **route** first, then by heavy, rarely used widgets (charts, editors, modals). → *More:* §10.8 Code splitting with `lazy` and `<Suspense>` · Ch 15 Q37

Q162. Does `lazy` support named exports? No — `lazy` expects a module with a **default export**. For a named export, map it: `lazy(() => import('./Charts').then(m => ({ default: m.LineChart })))`, or create a small re-export file. → *More:* §10.8 Code splitting with `lazy` and `<Suspense>`

Q163. ⋆ How do you render a list of 10,000 rows efficiently? What is virtualization (windowing)? Virtualization renders only the rows **visible** in the scroll area (plus a small buffer), positioning them inside a container sized for the full list, and swaps rows as you scroll. DOM stays small (~30 rows) no matter how long the list. Libraries: **TanStack Virtual**, react-window, react-virtuoso. Alternatives: pagination, "load more", or CSS `content-visibility: auto`. Give rows stable keys and fixed or measured heights. → *More:* §10.9 List virtualization · Ch 15 Q38

Q164. ⋆ What are debouncing and throttling? Where do you use them in React? Debounce: run only after events **stop** for N ms — search-as-you-type, autosave, resize end. **Throttle**: run **at most once** every N ms while events keep coming — scroll position, mouse move, infinite-scroll checks. In React, debounce a

value with `useDebounce`, or keep a debounced function stable in a ref/`useMemo` (a new debounced function each render resets its timer), and clear timers in cleanup. → *More:* §7.5 `useDebounce` — wait until the user stops

Q165. What does the React DevTools Profiler show? How do you find why a component re-rendered? The **Profiler** records commits and shows, per component, how long it took to render and (with "Record why each component rendered" on) **why** — props changed, state, hooks, parent rendered. Flame and ranked charts show the most expensive components. "Highlight updates when components render" visualises re-renders live. Programmatically, `<Profiler id onRender>` reports timings. → *More:* §10.3 Measure first: DevTools Profiler and `<Profiler>`

Q166. ⋆ What are common React performance bugs? Components defined inside components (remount every render); index keys on dynamic lists; new object/array/function props breaking `memo`; one giant context with frequently changing value; storing derived data in state + Effects (extra renders); fetching waterfalls; expensive work in render without memoization; huge lists without virtualization; importing whole libraries (lodash, moment) instead of what you use; memory leaks from missing cleanup. → *More:* §10.10 Common performance bugs

Q167. How do you optimise images, bundles and initial load (Core Web Vitals)? Core Web Vitals are Google's user-experience metrics: LCP (largest content shown), INP (responsiveness to input), CLS (layout shift). Improve them with SSR/SSG or streaming for faster first content, responsive images with width/height set (no layout shift) and `loading="lazy"` below the fold, `preload` of critical resources, code splitting and tree-shaking, analysing bundles (e.g. `rollup-plugin-visualizer`), transitions for heavy updates (INP), and caching/CDNs. → *More:* §10.11 A performance checklist · §13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR

Q168. What is tree-shaking? Tree-shaking is the bundler removing code that is never imported/used, based on static ES module `import`/`export` analysis. It works best with ESM libraries and named imports (`import { debounce } from 'lodash-es'`). CommonJS modules and side-effectful files are hard to shake. → *More:* §10.8 Code splitting with `lazy` and `<Suspense>`

Q169. When should you NOT use `memo`/`useMemo`/`useCallback`? When the component or calculation is cheap, when props change on almost every render anyway, when nothing compares the reference, or when the React Compiler already handles it. Memoization costs memory and comparison time and makes code harder to read; add it where the Profiler shows a real problem. → *More:* §10.4 `memo` — skipping a child's re-render · §10.7 React Compiler

---

11. Error boundaries, portals, patterns and accessibility

Q170. ⋆ What are error boundaries, and what don't they catch? An **error boundary** is a component that catches errors thrown while **rendering** its children (and in their lifecycle methods/constructors) and shows a fallback UI instead of a blank screen. They do **not** catch errors in **event handlers**, **async code** (`setTimeout`, promise callbacks outside of React's render), **server-side rendering**, or errors in the boundary itself. Handle those with `try/catch` and state — or rethrow into React via `setState(() => { throw err })`; Action errors in transitions do reach boundaries. → *More:* §14.9 Error boundaries · Ch 15 Q48

Q171. ⋆ How do you write an error boundary? Can it be a function component? It must be a **class** implementing `static getDerivedStateFromError(error)` (return state that shows the fallback) and/or `componentDidCatch(error, info)` (log it — `info.componentStack` shows where). There's no Hook equivalent; most apps use the **`react-error-boundary`** package (`<ErrorBoundary FallbackComponent={…} onReset={…}>`), which also offers `useErrorBoundary` to show async errors.

```
class Boundary extends React.Component {
  state = { error: null };
  static getDerivedStateFromError(error) { return { error }; }
  componentDidCatch(error, info) { logError(error, info.componentStack); }
  render() { return this.state.error ? this.props.fallback : this.props.children; }
}
```

→ *More:* §14.9 Error boundaries

Q172. Where should you place error boundaries? At several levels: one at the **top** (a "something went wrong" page so the app never shows a white screen), one per **route**, and around independent **widgets** (a chart, a comments panel) so one failure doesn't take down the page. Give users a way to **retry** (reset the boundary, e.g. with a `key` or `resetKeys`). → *More:* §14.9 Error boundaries

Q173. ⋆ What is a portal? How do events behave inside one? `createPortal(children, domNode)` renders children into a **different DOM node** (often `document.body`) while keeping them in the same place in the **React tree**. Used for modals, tooltips, dropdowns and toasts that must escape `overflow: hidden` or `z-index` stacking. Events from inside the portal **bubble through the React tree** (to React ancestors), not the DOM tree, and context still works. → *More:* §14.8 Portals · Ch 15 Q49

Q174. ⋆ What are Higher-Order Components (HOCs)? A **HOC** is a function that takes a component and returns a new component with extra behaviour: `const withAuth = (Comp) => (props) => useAuth().user ? <Comp {...props}/> : <Login/>`. Historic examples: `connect` from react-redux, `withRouter`. Drawbacks: wrapper nesting, prop-name collisions, hidden props and harder typing. Custom Hooks replace most of them today. → *More:* §14.6 Higher-order components (HOCs)

Q175. What are render props? A component takes a **function as a prop** (often `children`) and calls it to decide what to render, passing its internal state: `<MouseTracker>{({ x, y }) => <Dot x={x} y={y}/>}</MouseTracker>`. It shares logic while letting the caller control the UI. Mostly replaced by custom Hooks, still used by some libraries (e.g. list virtualizers, form libraries). → *More:* §14.5 Render props

Q176. ⋆ What are compound components? A set of components that work together and share implicit state through **Context**, giving a flexible, readable API: `<Tabs><Tabs.List><Tabs.Tab>…</Tabs.Tab></Tabs.List><Tabs.Panel>…</Tabs.Panel></Tabs>`. The parent owns state; children read it via context. Used by design systems (Radix, Headless UI, Reach) for tabs, menus, accordions, selects. → *More:* §14.4 Compound components

Q177. What is the container/presentational pattern? Is it still relevant? Container components handle data and logic (fetching, state); **presentational** components only receive props and render UI. It separates concerns and makes UI easy to test and reuse in Storybook. Today the "container" part is usually a **custom Hook** (or a Server Component / loader) rather than a wrapper component, but the idea — keep UI components dumb — still applies. → *More:* §14.2 Container / presentational

Q178. What are some React anti-patterns? Mutating state or props; copying props into state; Effects for derived data or event logic; index keys on dynamic lists; components defined inside components; giant components and "God" contexts; prop drilling through many layers; `useEffect` chains; disabling the exhaustive-deps lint rule; overusing `memo` everywhere without measuring; `dangerouslySetInnerHTML` with unsanitised input; deeply nested state that's hard to update. → *More:* §10.10 Common performance bugs · §5.6 You might not need an Effect

Q179. ⋆ How do you make React components accessible (a11y)? Use **semantic HTML** first (`<button>`, `<a href>`, `<label>`, `<nav>`, headings in order) — it brings keyboard support and screen-reader roles for free. Label every input (`<label htmlFor>` with `useId`), give images `alt`, keep visible **focus** and logical tab order, manage focus in modals (trap, restore), use ARIA only to fill gaps (`aria-expanded`, `aria-live` for updates, `aria-invalid`), ensure colour contrast, and test with keyboard, a screen reader, `eslint-plugin-jsx-a11y` and axe. Testing Library's role queries push you toward accessible markup. → *More:* §14.10 Accessibility in React · Ch 15 Q52

Q180. How do you manage focus in a modal dialog? On open, move focus into the dialog (the first focusable element or the dialog itself); **trap** Tab/Shift+Tab inside; close on Escape; on close, **return focus** to the element that opened it. Mark the rest of the page `inert` (or use the native `<dialog>` element with `showModal()`, which does most of this). Add `role="dialog"`, `aria-modal="true"` and `aria-labelledby`. → *More:* §15.13 Build 4 — Modal dialog (portal, focus trap, Escape)

Q181. How do you announce dynamic updates to screen readers? Use a **live region**: an element with `aria-live="polite"` (or `role="status"`) that exists in the DOM before the update; when its text changes, screen readers read it out. Use `role="alert"` (assertive) for errors. Typical uses: "3 results found", "Saved", toast messages. → *More:* §14.10 Accessibility in React

Q182. How do you structure a large React project? Group by **feature** (`features/cart/` with its components, hooks, API calls, tests) rather than by file type (`components/`, `hooks/`), plus a `shared/` or `ui/` folder for design-system components and an `app/` folder for routing and providers. Keep a public `index`

per feature, avoid deep cross-feature imports, colocate tests and styles, and let the framework's file routing define `app/` or `routes/`. → *More:* §14.11 Folder structure

Q183. What are the ways to style React components, and how do you choose? Plain CSS / Sass with naming conventions, **CSS Modules** (locally scoped class names), **utility CSS** (Tailwind), **CSS-in-JS** (styled-components, Emotion — runtime cost and friction with Server Components; zero-runtime options like vanilla-extract), inline `style` (dynamic values only), and component libraries (MUI, Chakra, shadcn/ui on Radix). Choose by team skills, Server Component compatibility, theming needs and performance. → *More:* §8.1 The styling landscape · §8.10 Choosing an approach

Q184. How do you implement a dark/light theme toggle? Store the choice in state (initialised from `localStorage` or `prefers-color-scheme`), set a `data-theme` attribute or class on `<html>`, and define colours as **CSS custom properties** per theme so no component re-renders for colours. Persist on change. With SSR, set the attribute in a tiny inline script before paint to avoid a flash of the wrong theme. → *More:* §8.8 Theming and dark mode

Q185. How do you localise (internationalise) a React app? Use a library — **react-i18next**, **FormatJS/react-intl**, or Lingui — that loads translation files per language and provides `t('key')` or `<FormattedMessage>`; format dates, numbers and currencies with the built-in `Intl` APIs; handle plurals and right-to-left languages (`dir="rtl"`); keep the locale in the URL or a cookie so SSR renders the right language; lazy-load translation bundles. → *More:* §6.3 Context — pass data deeply without props

---

12. SSR, SSG, hydration and Next.js

Q186. ⋆ CSR vs SSR vs SSG vs ISR — what's the difference? **CSR** (client-side rendering): the server sends an almost empty HTML page plus JavaScript, which builds the UI in the browser — simple hosting, slow first paint, weak SEO. **SSR** (server-side rendering): the server renders HTML **per request**, then the browser hydrates it — fast first content and fresh data, costs server work. **SSG** (static site generation): HTML rendered **at build time** and served from a CDN — fastest, but data is as old as the last build. **ISR** (incremental static regeneration, Next.js): static pages **re-generated in the background** after a time or on demand. Streaming and partial prerendering mix these per part of a page. → *More:* §13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR · Ch 15 Q43

Q187. ⋆ What is hydration? **Hydration** is React, in the browser, taking over HTML that was rendered on the server: it renders the same components, matches them to the existing DOM nodes instead of creating new ones, and **attaches event handlers and state** so the page becomes interactive. Use `hydrateRoot(container, <App/>)`. Until hydration finishes, the page is visible but clicks don't do anything. → *More:* §13.4 Hydration end to end · Ch 15 Q44

Q188. ⋆ What causes a hydration mismatch, and how do you fix it? A mismatch happens when the client's first render produces different HTML than the server's: using `Date.now()`, `Math.random()`, `window`/`localStorage`, the user's locale or time zone during render, invalid HTML nesting (`<div>` inside `<p>`), or browser extensions changing the DOM. Fixes: render the same output on both sides and switch to client-only values **after** hydration (in an Effect, or `useSyncExternalStore` with a server snapshot), use `useId` for IDs, and `suppressHydrationWarning` only for unavoidable one-element differences such as a timestamp. → *More:* §13.5 Hydration mismatches

Q189. What is streaming SSR and selective hydration? With **streaming** (`renderToPipeableStream` in Node, `renderToReadableStream` for Web streams), the server sends HTML in chunks: the shell first, and each `<Suspense>` boundary's content when its data is ready (with the fallback shown meanwhile). **Selective hydration** lets React hydrate parts independently and prioritise the part the user clicks. Result: fast first paint without waiting for the slowest data. `renderToString` can't wait for data or stream. → *More:* §13.3 Streaming SSR with `<Suspense>` · §13.2 The server rendering APIs

Q190. What is partial prerendering (PPR)? A page is split into a **static shell** prerendered at build time (served instantly from a CDN) and **dynamic holes** inside `<Suspense>` boundaries that are rendered and streamed per request. You get static speed for most of the page and fresh, personalised data where needed. React provides the primitives (`prerender`, `resume`); Next.js 16 enables it with Cache Components. → *More:* §13.6 Static generation and partial prerendering

Q191. ⋆ What is Next.js, and why use it? **Next.js** is a React framework: file-based routing, server rendering (SSR, SSG, ISR, streaming, PPR), React Server Components and Server Functions, API route handlers, image/font optimisation, bundling (Turbopack) and deployment integrations. You get SEO-friendly, fast-loading pages and full-stack features without wiring them yourself. React's docs recommend starting new apps with a framework like it. → *More:* §13.9 Next.js App Router overview

Q192. ⋆ Next.js App Router vs Pages Router? The **Pages Router** (`pages/`) uses `getServerSideProps` / `getStaticProps` for data and treats every component as a client component. The **App Router** (`app/`, since Next.js 13) is built on **Server Components** by default, nested **layouts**, `loading.js` / `error.js` files (Suspense and error boundaries per segment), streaming and Server Functions; you fetch with plain `async` / `await` in components. New features go to the App Router; Pages is still supported. → *More:* §13.9 Next.js App Router overview

Q193. What are the special files in the Next.js App Router? In each route folder: `page.js` (the route's UI, makes it reachable), `layout.js` (shared UI that **persists** across child navigations), `loading.js` (Suspense fallback), `error.js` (error boundary — a client component), `not-found.js`, `route.js` (an API handler for GET/POST…), `template.js` (like layout but remounts). Dynamic segments use folders like `[id]`; `params` and `searchParams` are **Promises** since Next.js 15 (`await params`), and Next.js 16 removed the old synchronous access. At the project root, `proxy.ts` (renamed from `middleware.ts` in Next.js 16) intercepts requests before routing. → *More:* §13.9 Next.js App Router overview

Q194. How do caching and revalidation work in Next.js 16? By default (without Cache Components), pages are static unless they read dynamic data; `export const revalidate = 60` or `fetch(url, { next: { revalidate: 60 } })` gives ISR. With **Cache Components** (`cacheComponents: true`), everything is dynamic unless you mark it with the `'use cache'` directive (with `cacheLife` / `cacheTag`); dynamic parts must sit in `<Suspense>`. After a mutation, call `revalidatePath`, `revalidateTag(tag, 'max')` (Next.js 16 requires a `cacheLife` profile as the second argument; it serves stale data while refetching in the background) or, inside a Server Action, `updateTag(tag)` for read-your-own-writes; `refresh()` re-renders only uncached data. → *More:* §13.9 Next.js App Router overview

Q195. ⋆ What goes wrong if you put `'use client'` at the top of the root layout? `'use client'` marks a **boundary**: that module and everything it imports become Client Components. At the root, the whole app ships as client JavaScript, so you lose Server Components' benefits — server data fetching in components, secrets on the server, zero-JS rendering. Keep layouts as Server Components and put `'use client'` on small interactive leaves; providers go in a small client `Providers` component that wraps `children`. → *More:* §13.8 Directives: `'use client'` and `'use server'` · Ch 15 Q47

Q196. How do you do SEO in a React app? Render real HTML on the server (SSR/SSG) so crawlers and link previews see content; set a unique `<title>`, meta description and Open Graph tags per page (React 19 hoists `<title>` / `<meta>` rendered anywhere; Next.js has the `metadata` export / `generateMetadata`); use semantic headings and links (`<a href>`), clean URLs, a sitemap and `robots.txt`, and good Core Web Vitals. → *More:* §13.1 Rendering strategies: CSR, SSR, SSG, ISR, streaming, PPR · §13.9 Next.js App Router overview

Q197. What is `window is not defined` during SSR, and how do you avoid it? On the server there is no `window`, `document` or `localStorage`; code that touches them during render or at module top level crashes SSR. Access browser APIs inside **Effects or event handlers** (they run only on the client), guard with `typeof window !== 'undefined'` for module code, or load a browser-only component lazily with SSR disabled (`next/dynamic` with `ssr: false` in a Client Component). → *More:* §13.5 Hydration mismatches

Q198. When would you choose a plain SPA (Vite) over Next.js? For apps behind a login where SEO doesn't matter (admin panels, dashboards, internal tools), apps embedded in another site, or teams that want static hosting with no server. Choose a framework when you need SEO, fast first load for public pages, server data access or Server Components. React Router's framework mode is the middle ground (SPA, SSR or prerender per route). → *More:* §13.11 Choosing a framework

---

13. Testing and TypeScript

Q199. ⋆ How do you test React applications? **Unit/component tests** with **Vitest** or **Jest** plus **React Testing Library** (render components, interact like a user, assert on what's visible); **mock network** with MSW

(Mock Service Worker) rather than mocking `fetch` per test; **custom Hooks** with `renderHook`; **end-to-end tests** of real user flows in a real browser with **Playwright** or Cypress; plus TypeScript, ESLint and visual tests (Storybook) as cheap safety nets. Aim for most value from integration-style component tests. → *More:* §12.1 What to test, and the Testing Library philosophy · §12.10 Good tests, bad tests, and the bigger picture

Q200. ⋆ What is the philosophy of React Testing Library? Why not Enzyme? "The more your tests resemble the way your software is used, the more confidence they can give you." Query by what users see — **role, label, text** — not by class names or component internals; interact with `user-event`; assert visible results. Tests then survive refactors. **Enzyme** tested implementation details (state, instance methods, shallow rendering) and has no React 18/19 support. → *More:* §12.1 What to test, and the Testing Library philosophy · Ch 15 Q51

Q201. ⋆ What is the priority of queries, and what's the difference between `getBy`, `queryBy` and `findBy`? Priority: `getByRole` (with `name`) → `getByLabelText` → `getByPlaceholderText` → `getByText` → `getByDisplayValue` → `getByAltText` / `getByTitle` → `getByTestId` as last resort. `getBy*` **throws** if not found (use for things that must be there); `queryBy*` returns `null` (use to assert absence); `findBy*` returns a **Promise** that retries until found or timeout (use for things that appear asynchronously). `*AllBy*` variants return arrays. → *More:* §12.3 Queries and their priority

Q202. How do you test async behaviour (data loading)? Mock the network (MSW or a stubbed `fetch`), render, then `await screen.findByText('Alice')` or `await waitFor(() => expect(...))` for the result. Assert the loading state first if relevant, and the error state with a failing mock. Don't use arbitrary `setTimeout` sleeps; for timers use fake timers (`vi.useFakeTimers()`) and advance them. → *More:* §12.5 Async tests · §12.6 Mocking `fetch` (and when to use MSW)

Q203. `fireEvent` vs `user-event`? `fireEvent.click(el)` dispatches a single DOM event. `user-event` (`const user = userEvent.setup(); await user.type(input, 'hi')`) **simulates real interactions** — focus, keydown, keypress, input, keyup for each character, pointer events, disabled checks — so it catches more bugs. Prefer `user-event`; its methods are async. → *More:* §12.4 `user-event` — interacting like a user

Q204. What is `act()`, and why do I see "not wrapped in act(...)" warnings? `act()` makes sure all state updates, Effects and re-renders caused by some code are **finished** before you assert. Testing Library's `render`, `fireEvent`, `user-event` and `findBy` / `waitFor` already wrap in `act`. The warning usually means an update happened **after** your test finished looking — e.g. a fetch resolved later; fix it by awaiting the result (`findBy…`) rather than wrapping things manually. In React 19 `act` is imported from `react`. → *More:* §12.5 Async tests

Q205. How do you test a custom Hook? Components that need providers or a router? Hooks: `renderHook(() => useCounter())` from `@testing-library/react`, then `act(() => result.current.increment())` and check `result.current`. Or test through a small component that uses it. Providers/routers: pass a `wrapper` (`render(ui, { wrapper: Providers })`) or write a **custom render** that wraps with `QueryClientProvider`, store and `MemoryRouter` / `createMemoryRouter`. → *More:* §12.7 Testing custom Hooks · §12.8 Providers, routers and custom render

Q206. Are snapshot tests a good idea? Snapshots save the rendered output and fail when it changes. They catch accidental changes cheaply but are often **approved blindly**, break on any markup tweak, and don't tell you what behaviour matters. Use small, focused snapshots (e.g. a serialised data structure) sparingly; prefer explicit assertions about what the user sees. → *More:* §12.10 Good tests, bad tests, and the bigger picture

Q207. What should you test, and what not? Test **behaviour** that users and other code depend on: rendering for given props, interactions, form validation, loading/error/empty states, accessibility of key flows. Don't test implementation details (state variable names, which Hook is used, CSS classes), third-party libraries, or trivial markup. Write a test for every bug you fix. → *More:* §12.10 Good tests, bad tests, and the bigger picture

Q208. ⋆ How do you type component props, children and events in TypeScript? Props with a `type` / `interface` and destructuring: `function Button({ label, onClick }: { label: string; onClick: () => void })`. Children: `children: React.ReactNode`. Events: `React.ChangeEvent<HTMLInputElement>`, `React.MouseEvent<HTMLButtonElement>`, `React.FormEvent<HTMLFormElement>`; or let inference work by writing the handler inline. Extend native props with `React.ComponentProps<'button'>`. Avoid `React.FC` for most

components (it used to add implicit `children`). → *More:* §11.2 Typing props · §11.4 Typing events · Ch 15 Q53

Q209. How do you type `useState`, `useRef`, `useReducer` and Context? `useState<User | null>(null)` when the initial value doesn't show the full type. `useRef<HTMLInputElement>(null)` for DOM refs (React 19 types require an argument); `useRef<number | null>(null)` for timer IDs. Reducers: type state and a **discriminated union** of actions (`{ type: 'added'; text: string } | { type: 'removed'; id: number }`), and TypeScript narrows in each `case`. Context: `createContext<CartCtx | null>(null)` plus a custom Hook that throws if null, so consumers get a non-null type. → *More:* §11.5 Typing state and reducers · §11.6 Typing refs · §11.7 Typing custom Hooks and Context

Q210. What are generic components, and when do you need them? A **generic component** has a type parameter so its props relate to each other: a `List<T>` takes `items: T[]` and `renderItem: (item: T) => ReactNode`, and TypeScript checks `renderItem` receives the right item type. Write it as `function List<T>({ items, renderItem }: ListProps<T>)` (in `.tsx`, arrow functions need `<T,>`). Used for tables, selects, autocompletes. → *More:* §11.8 Generic components

Q211. How do you model props where some combinations are invalid? Use **discriminated unions**: `type Props = { variant: 'link'; href: string } | { variant: 'button'; onClick: () => void }`. TypeScript then rejects `variant="link"` without `href`, and narrows inside the component when you check `props.variant`. Better than many optional props that may or may not go together. → *More:* §11.9 Discriminated unions for props

Q212. `interface` vs `type` for props? What are the benefits of TypeScript with React? Both work for props; `interface` can be extended/merged and gives slightly clearer errors, `type` supports unions and mapped types — pick one convention. Benefits of TypeScript: catches wrong/missing props and typos at compile time, autocompletion and safe refactoring across a codebase, self-documenting component APIs, and typed API responses and context. It replaces PropTypes. → *More:* §11.1 Setup and how type-checking fits in · §11.2 Typing props

---

14. "What will this code print?" — output and behaviour puzzles

Q213. ⋆ What does the button show after one click?

```
const [n, setN] = useState(0);
<button onClick={() => { setN(n + 1); setN(n + 1); setN(n + 1); }}>{n}</button>
```

1. All three calls read the same snapshot `n = 0` and set 1. With `setN(x => x + 1)` three times it would show **3**. `setN(n + 5); setN(x => x + 1)` gives **6** (replace with 5, then add 1). → *More:* §3.5 Updater functions — queueing updates · Ch 15 Q8

Q214. ⋆ What is logged?

```
function handleClick() { setCount(count + 1); console.log(count); }
```

The **old** value (e.g. `0` on the first click). The setter schedules a re-render; the local `count` is a constant for this render. → *More:* §3.3 State as a snapshot

Q215. What does this render when `items` is empty: `{items.length && <List items={items} />}`? A literal `0`. `0 && …` evaluates to `0`, and React renders numbers. `false`, `null`, `undefined` and `true` render nothing; `0` and `NaN` render as text. Use `items.length > 0 && …` or a ternary. → *More:* §2.7 Conditional rendering · Ch 15 Q7

Q216. ⋆ What is logged, and in what order?

```
function Child() { console.log('child render'); useEffect(() => console.log('child effect')); return null; }
function Parent() { console.log('parent render'); useEffect(() => console.log('parent effect')); return <Child />;
}
```

`parent render`, `child render`, `child effect`, `parent effect`. Rendering is top-down; Effects run **bottom-up** after commit. (Under StrictMode in development each component body runs twice — `parent render` ×2, `child render` ×2 — and then the Effects run, are cleaned up and run again: `child effect`, `parent effect`, `child effect`, `parent effect`.) → *More:* §5.3 Cleanup — undo what setup did · Ch 15 Q18

Q217. What does this counter show after 5 seconds?

```
const [count, setCount] = useState(0);
useEffect(() => {
  const id = setInterval(() => setCount(count + 1), 1000);
  return () => clearInterval(id);
}, []);
```

1, forever. The interval callback closed over the first render's `count = 0`, so it keeps setting 1 (a stale closure). Fix: `setCount(c => c + 1)`. (And with no cleanup, StrictMode in development would start two intervals.) → *More:* §5.5 Dependencies in depth

Q218. ⋆ Why does this component re-render endlessly?

```
function Profile({ id }) {
  const [user, setUser] = useState(null);
  const options = { id };
  useEffect(() => { fetchUser(options).then(setUser); }, [options]);
}
```

`options` is a **new object** every render; the Effect runs, sets state, re-renders, creates a new `options`, runs again … Depend on `id` and build the object inside the Effect. → *More:* §5.5 Dependencies in depth

Q219. What happens here?

```
function Counter() {
  const [n, setN] = useState(0);
  setN(n + 1);
  return <p>{n}</p>;
}
```

React throws **"Too many re-renders. React limits the number of renders to prevent an infinite loop."** Setting state unconditionally during render schedules another render, forever. The same happens with `onClick={setN(n + 1)}` (called during render instead of passed). → *More:* §3.1 Responding to events · §16.18 Common errors and fixes

Q220. Does this list update when you click "Add"?

```
const [items, setItems] = useState([]);
const add = () => { items.push('x'); setItems(items); };
```

No. `items` was mutated and the **same array reference** passed back; `Object.is(old, new)` is true, so React skips the render. Use `setItems([...items, 'x'])`. → *More:* §3.6 Updating objects and arrays immutably

Q221. ⋆ A user types in the first row's input, then you add an item at the top of a list keyed by index. What happens? The typed text now appears in the **new** first row instead of staying with its original item. With index keys, the old row 0's component (and its uncontrolled input state) is reused for whatever is now at index 0. Stable IDs as keys fix it. → *More:* §2.8 Rendering lists and keys · Ch 15 Q4

Q222. What does this log in development, and why?

```
function App() { console.log('render'); return null; }
createRoot(root).render(<StrictMode><App /></StrictMode>);
```

`render` logged **twice** in development (React DevTools dims the second log), once in production. StrictMode double-invokes component bodies to expose impure rendering. → *More:* §5.4 StrictMode's double run (development only)

Q223. In what order are these logged on mount: `useEffect`, `useLayoutEffect`, render? `render` → `layout effect` → (browser paints) → `effect`. Layout effects run synchronously after DOM mutation and before paint; passive Effects run after paint. → *More:* §5.11 `useLayoutEffect` — before the browser paints · §16.5 The render and commit lifecycle

Q224. How many renders happen after clicking this button (React 18+)?

```
onClick={() => { setTimeout(() => { setA(1); setB(2); setC(3); }, 0); }}
```

One re-render. Since React 18, automatic batching groups updates inside timeouts, promises and native handlers too (in React 17 it would have been three). → *More:* §3.4 Batching

Q225. What happens to the input text when `isEditing` toggles?

```
return isEditing ? <input key="a" /> : <input key="b" />;
```

The text is **lost** — different keys mean different components, so React unmounts one input and mounts a fresh one. Without the keys (same type, same position) the DOM input and its text would be **kept**. → *More:* §3.10 State is tied to a position in the tree

Q226. ⋆ Does `Child` re-render when `Parent`'s state changes?

```
const Child = memo(function Child({ onClick }) { … });
function Parent() { const [n, setN] = useState(0); return <Child onClick={() => setN(n + 1)} />; }
```

Yes, every time: the arrow function is a new reference each render, so `memo`'s shallow comparison fails. Wrap the handler in `useCallback` (with `setN(x => x + 1)` so deps can be `[]`), or rely on the React Compiler. → *More:* §10.4 `memo` — skipping a child's re-render · §10.5 `useCallback` and `useMemo` for stable props

Q227. What is rendered by `{[1, 2, 3]}`, `{true}`, `{null}`, `{undefined}`, `{{ a: 1 }}`? `123` (arrays render each item — React warns about missing keys only for arrays of elements); nothing for `true`, `null` and `undefined`; and `{{ a: 1 }}` **throws** "Objects are not valid as a React child" — render a field (`obj.a`) or `JSON.stringify(obj)`. → *More:* §1.8 JSX · §16.18 Common errors and fixes

Q228. A ref counter is incremented on click. Does the screen update?

```
const clicks = useRef(0);
<button onClick={() => { clicks.current++; }}>Clicked {clicks.current}</button>
```

No — changing a ref doesn't trigger a render, so the text stays at 0 until something else re-renders the component (then it jumps to the current number). Use state for anything displayed. → *More:* §5.8 `useRef` for values

---

15. Machine coding and scenario questions

These are the prompts most often given in 45–90-minute "machine coding" rounds. The answer gives the **core state and key idea** you should say before coding; the linked build in Chapter 15 has the full code and tests. Approach for every one: clarify requirements → sketch component tree and state → build the happy path → add edge cases (empty, loading, error, keyboard, a11y) → mention what you'd do with more time.

Q229. ⋆ Build a todo app (add, toggle, delete, edit, filter all/active/completed). State: `todos: { id, text, done }[]` and `filter`. Visible todos are **derived** (`todos.filter(...)`), not stored. Add with a generated `id` (`crypto.randomUUID()`), toggle/edit with `map`, delete with `filter` — all immutable; key rows by `id`. Use a `<form>` so Enter submits, trim/ignore empty text, and a reducer once actions multiply. Extras: persist to `localStorage`, "clear completed", item count. → *More:* §15.10 Build 1 — Todo list with filters · §3.11 Putting it together — a to-do app

Q230. ⋆ Build an autocomplete / typeahead search. State: `query`, `results`, `status`, `activeIndex`, `open`. Debounce the query (≈300 ms), fetch with `AbortController` and abort the previous request in cleanup (no stale results), cache results by query, handle empty/loading/error/no-results. Keyboard: ArrowUp/Down move `activeIndex`, Enter selects, Escape closes; ARIA `combobox`/`listbox`/`option` with `aria-activedescendant`; highlight the matched text; close on outside click. → *More:* §15.11 Build 2 — Autocomplete (debounce, abort, keyboard) · §7.5 `useDebounce` — wait until the user stops

Q231. ⋆ Implement infinite scroll. State: `items`, `page`/`cursor`, `hasMore`, `loading`, `error`. Put a **sentinel** `<div>` after the list and watch it with an `IntersectionObserver` in an Effect (disconnect in

cleanup); when it becomes visible and not already loading and `hasMore`, fetch the next page and append. Guard against duplicate requests (a ref flag), show a loader/error with retry, and virtualise if the list gets huge. TanStack Query's `useInfiniteQuery` handles pages/cursors.

```
useEffect(() => {
  const io = new IntersectionObserver(([e]) => { if (e.isIntersecting) loadMore(); });
  io.observe(sentinelRef.current);
  return () => io.disconnect();
}, [loadMore]);
```

→ *More:* §15.12 Build 3 — Infinite scroll

Q232. ⋆ Build pagination. State: `page` (ideally in the URL search params), `pageSize`; `totalPages = Math.ceil(total / pageSize)`. Render Prev/Next (disabled at the ends), page numbers with ellipses for many pages (first, last, current ± 1), `aria-current="page"`. Client-side: slice the array; server-side: fetch `?page=&limit=` and keep the previous page visible while loading (`placeholderData: keepPreviousData` in TanStack Query). Clamp page when filters reduce the total. → *More:* §15.16 Build 7 — Pagination · §9.9 Query keys, pagination and dependent queries

Q233. ⋆ Build a modal dialog. Props: `open`, `onClose`, `title`, `children`. Render through a **portal** to `document.body`; overlay click and **Escape** close it; **focus** moves inside on open, Tab is trapped, focus returns to the trigger on close; body scroll locked; `role="dialog"`, `aria-modal`, `aria-labelledby`. Or use the native `<dialog>` with `showModal()` and a ref. → *More:* §15.13 Build 4 — Modal dialog (portal, focus trap, Escape) · §14.8 Portals

Q234. ⋆ Build tabs. State: `activeIndex` (or ID). Buttons with `role="tab"`, `aria-selected`, `aria-controls`; panels with `role="tabpanel"`; arrow keys move between tabs (wrap around), Home/End jump; only the active tab is in the Tab order (`tabIndex` 0 vs −1). A compound-component API (`<Tabs.List>`, `<Tabs.Panel>`) via Context is a nice touch. Decide whether inactive panels unmount (lose state) or stay hidden (`<Activity>` or `hidden`). → *More:* §15.14 Build 5 — Accessible tabs · §14.4 Compound components

Q235. ⋆ Build an accordion. State: `openIds` — a `Set` (multiple open) or a single ID (one at a time; make it a prop option). Each header is a `<button aria-expanded={open} aria-controls={panelId}>` inside a heading; panel has `id` and `role="region"`. Toggle by creating a **new** Set. Simplest accessible version: native `<details>` / `<summary>`. Animate height with CSS `grid-template-rows` or `interpolate-size`.

```
const toggle = (id) => setOpen(prev => {
  const next = new Set(prev); next.has(id) ? next.delete(id) : next.add(id); return next;
});
```

→ *More:* §15.18 Other common prompts — which techniques they need

Q236. ⋆ Build a star rating component. Props: `value`, `onChange`, `max = 5`; state: `hover` (preview). Render `max` stars; filled if `i < (hover ?? value)`; click sets value, `mouseleave` clears hover; support keyboard (it's a radio group: `role="radiogroup"` with `role="radio"` / `aria-checked`, or real `<input type="radio">`s with labels), optional half stars and read-only mode. → *More:* §15.15 Build 6 — Star rating

Q237. ⋆ Build nested comments (Reddit-style thread). Data: either a tree (`{ id, text, replies: [] }`) or — better for updates — a **flat map by ID** with `parentId`, building children lists. Render with a **recursive** `<Comment>` that renders its replies (keyed by ID), indented. Features: reply (insert under parent), edit, delete (soft-delete if it has replies), collapse/expand, votes, sort. Updates on a tree need a recursive immutable update; on a flat map they're O(1).

```
function Comment({ id, byId }) {
  const c = byId[id];
  return <li>{c.text}<ul>{c.childIds.map(cid => <Comment key={cid} id={cid} byId={byId} />)}</ul></li>;
}
```

→ *More:* §15.18 Other common prompts — which techniques they need

Q238. ⋆ Build a file explorer (folder tree). Data: a tree of `{ id, name, isFolder, children }`. Recursive `<Node>` component; state `expanded: Set<id>` (folders toggle on click), plus add file/folder, rename and

delete via immutable recursive updates (or a normalised map). Show icons, sort folders first, keyboard navigation with ARIA `tree` / `treeitem` / `aria-expanded`. Talk about lazy-loading children for huge trees.

```
function insert(node, parentId, item) {
  if (node.id === parentId) return { ...node, children: [...node.children, item] };
  if (!node.children) return node; // a file: nothing to search
  return { ...node, children: node.children.map(c => insert(c, parentId, item)) };
}
```

→ *More:* §15.18 Other common prompts — which techniques they need

Q239. ⋆ Build a countdown timer / stopwatch. State: `status` (idle/running/paused), `endAt` or `startedAt` timestamp, `remaining`. Start a `setInterval` in an Effect **only while running**, clear it in cleanup; compute remaining from `Date.now()` each tick instead of decrementing (intervals drift and pause in background tabs). Start/pause/resume/reset; stop at zero and fire `onComplete` once; format `mm:ss`. Keep the interval ID in a ref if you start it from handlers.

```
useEffect(() => {
  if (status !== 'running') return;
  const id = setInterval(() => setNow(Date.now()), 250);
  return () => clearInterval(id);
}, [status]);
const remaining = Math.max(0, endAt - now);
```

→ *More:* §15.18 Other common prompts — which techniques they need · §5.3 Cleanup — undo what setup did

Q240. ⋆ Build a form with validation (sign-up form, multi-step wizard). State: `values`, `errors`, `touched`, `status`. Validate per field on blur and all on submit with a pure `validate(values)` (or a Zod schema); disable submit while submitting; show errors next to fields with `aria-invalid` and `aria-describedby`; focus the first invalid field. Multi-step: one `values` object across steps, validate the current step before Next, keep Back working, review step before submit. React 19 alternative: `<form action>` + `useActionState` returning field errors. → *More:* §15.17 Build 8 — Multi-step form wizard with validation · §4.3 Validation

Q241. ⋆ Write a `useFetch` custom Hook. Return `{ data, error, loading }` (a status enum is cleaner); refetch when the URL changes; abort the old request in cleanup to avoid races; handle non-2xx responses as errors.

```
function useFetch(url) {
  const [state, setState] = useState({ status: 'loading' });
  useEffect(() => {
    const ac = new AbortController(); setState({ status: 'loading' });
    fetch(url, { signal: ac.signal })
      .then(r => { if (!r.ok) throw new Error(r.status); return r.json(); })
      .then(data => setState({ status: 'success', data }))
      .catch(error => { if (error.name !== 'AbortError') setState({ status: 'error', error }); });
    return () => ac.abort();
  }, [url]);
  return state;
}
```

Mention: in production use TanStack Query (caching, retries, dedupe). → *More:* §7.6 `useFetch` — loading, error, success, and no races

Q242. ⋆ Write a `useDebounce` Hook. Return a copy of the value that updates only after it stops changing for `delay` ms; each change clears the previous timer.

```
function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}
```

Variant: `useDebouncedCallback(fn, delay)` keeps the timer in a ref and the latest `fn` in a ref. → *More:* §7.5 `useDebounce` — wait until the user stops

Q243. Write a `useThrottle` Hook. Allow the value through at most once per `interval`; schedule a trailing update so the final value isn't lost.

```
function useThrottle(value, ms = 200) {
  const [v, setV] = useState(value); const last = useRef(0);
  useEffect(() => {
    const wait = Math.max(0, last.current + ms - Date.now());
    const t = setTimeout(() => { last.current = Date.now(); setV(value); }, wait);
    return () => clearTimeout(t);
  }, [value, ms]);
  return v;
}
```

→ *More:* §7.5 `useDebounce` — wait until the user stops

Q244. Write `useLocalStorage`. Like `useState`, but the initial value is read lazily from `localStorage` (with `try/catch` and `JSON.parse`) and every change is written back. Handle SSR (no `window`), quota/JSON errors, and optionally sync across tabs with the `storage` event (or `useSyncExternalStore`).

```
function useLocalStorage(key, initial) {
  const [v, setV] = useState(() => {
    try { const s = localStorage.getItem(key); return s ? JSON.parse(s) : initial; } catch { return initial; }
  });
  useEffect(() => { try { localStorage.setItem(key, JSON.stringify(v)); } catch {} }, [key, v]);
  return [v, setV];
}
```

→ *More:* §7.4 `useLocalStorage` — state that survives a reload

Q245. Write `useOnClickOutside` (close a dropdown when clicking elsewhere). Listen on `document` for `pointerdown`; if the target isn't inside `ref.current`, call the handler. Keep the latest handler in a ref (or `useEffectEvent`) so the listener isn't re-attached each render; remove it in cleanup. Also close on Escape and on focus leaving.

```
function useOnClickOutside(ref, handler) {
  const onOutside = useEffectEvent(handler);
  useEffect(() => {
    const listener = (e) => { if (ref.current && !ref.current.contains(e.target)) onOutside(e); };
    document.addEventListener('pointerdown', listener);
    return () => document.removeEventListener('pointerdown', listener);
  }, [ref]);
}
```

→ *More:* §7.7 Hooks that take callbacks — `useEffectEvent`

Q246. Write `useInterval` (Dan Abramov's pattern) and `useWindowSize` / `useMediaQuery`. `useInterval(callback, delay)`: keep the latest callback in a ref (or `useEffectEvent`), start `setInterval` in an Effect keyed on `delay`, pause when `delay` is `null`. `useWindowSize` / `useMediaQuery`: subscribe to `resize` / `matchMedia` change events — best with `useSyncExternalStore(subscribe, () => window.matchMedia(q).matches, () => false)` so it's tear-free and SSR-safe. `getSnapshot` must return a primitive or a cached value — returning a fresh `{ width, height }` object each call causes an infinite re-render loop, so for `useWindowSize` read `window.innerWidth` and `window.innerHeight` with two separate subscriptions (or cache the object). → *More:* §7.9 `useSyncExternalStore` — subscribing to outside data · §7.7 Hooks that take callbacks — `useEffectEvent`

Q247. Build a toast/notification system. A `ToastProvider` holds a queue `{ id, message, type }[]` and exposes `toast.show()` via Context (or a store so it's callable outside React). Render toasts through a **portal**; each auto-dismisses with a timer cleared on unmount and paused on hover; limit how many show; announce with `role="status"` / `aria-live`; let users close them. → *More:* §15.18 Other common prompts — which techniques they need · §14.8 Portals

Q248. Build an image carousel/slider. State: `index`. Next/prev with wrap-around (`(i + 1) % n`), dots to jump, optional autoplay via `setInterval` in an Effect that pauses on hover/focus and when the tab is

hidden; preload the next image; keyboard arrows; `aria-roledescription="carousel"` and a live region for slide changes; swipe with pointer events on touch devices. → *More:* §15.18 Other common prompts — which techniques they need

Q249. Build an OTP (one-time password) input. State: an array of N digits; an array of refs (`useRef([])`). On input: keep the last digit, move focus to the next box; Backspace on an empty box moves back; paste spreads digits across boxes; only digits allowed (`inputMode="numeric"`); `autoComplete="one-time-code"` on the first box; call `onComplete` when all are filled. → *More:* §15.18 Other common prompts — which techniques they need

Q250. Build a progress bar. Props: `value` 0–100 (clamped). Render an outer track and an inner bar with `width: ${value}%` (or `transform: scaleX`) and a CSS transition; accessible with the native `<progress>` or `role="progressbar"` with `aria-valuenow/min/max`. Variants asked: animate from 0 to 100 over N seconds, or N bars filling one after another (a queue in state, each starting when the previous ends). → *More:* §15.18 Other common prompts — which techniques they need

Q251. Build a data table with sorting, filtering and search. State: `sort { key, dir }`, `filters`, `query`, `page`; rows derived with `filter` → `sort` (copy before sorting) → `slice`, memoized with `useMemo` for big data; debounce the search; keep state in the URL; clickable headers with `aria-sort`; virtualise or paginate large data; for server data, send these as query params instead. → *More:* §15.18 Other common prompts — which techniques they need · §10.6 `useMemo` for expensive calculations

Q252. Build tic-tac-toe (or a memory/match game). State: `squares` (array of 9), `xIsNext` (or derive from move count), optional `history` for time travel. `winner` is **derived** with a `calculateWinner` check over the 8 lines — don't store it. Ignore clicks on filled squares or after a win; show status (winner / draw / next player); reset button. Extension: N×N board, undo. → *More:* §3.8 Derived state — choosing the minimal state

Q253. Build a traffic light, or a "boxes turn green then reset in click order" grid. Traffic light: state `color`; an Effect sets a `setTimeout` for the current colour's duration from a config map (`{ red: 4000, green: 3000, yellow: 500 }`) and moves to `next`; clear the timer in cleanup. Click-order grid: store the **order** of clicked boxes in an array; when all are active, deactivate them one by one with timers in that order (store timer IDs in a ref to clean up on unmount). → *More:* §5.3 Cleanup — undo what setup did

Q254. Build a chips/tags input, a multi-select, or a drag-and-drop list (kanban). Chips: state `chips: string[]` and `input`; Enter adds a trimmed unique chip, Backspace on empty input removes the last, × button removes one. Multi-select: autocomplete + chips. Drag-and-drop: state as columns → ordered IDs (normalised), move with immutable splice logic; use `@dnd-kit` or native HTML5 drag events, plus a keyboard alternative. → *More:* §15.18 Other common prompts — which techniques they need

Q255. Build a "shopping cart" / checkout page. State: cart lines `{ productId, qty }` (normalised; product details looked up), derived subtotal/tax/total, quantity +/− with min 1 and stock max, remove, persist to `localStorage`, coupon validation. Shared across pages via Context + reducer or Zustand. Checkout form with validation; disable submit while paying; optimistic quantity changes with rollback. → *More:* §6.11 Putting it together — a bookshop

Q256. ⋆ Scenario: the page gets slower the longer it stays open. How do you investigate? Suspect a **memory leak**: listeners, intervals, subscriptions or observers not cleaned up; caches growing forever; detached DOM nodes kept by refs. Use the browser's Memory panel (heap snapshots before/after an action, compare), the Performance panel, and React DevTools Profiler for growing render times. Fix with Effect cleanups, bounded caches, and virtualization for growing lists. → *More:* §5.3 Cleanup — undo what setup did · §10.10 Common performance bugs

Q257. ⋆ Scenario: typing in a search box is laggy because it filters a large list. How do you fix it? Measure first. Then: keep the input state local and **defer** the expensive part — `useDeferredValue(query)` for the list, or `startTransition` around the filter update — so keystrokes stay urgent; memoize the filtered result (`useMemo`) and `memo` the list; **virtualise** the list; debounce if the filter hits the network; move heavy work to a Web Worker if it's CPU-bound. → *More:* §7.11 `useDeferredValue` — let part of the UI lag behind · §10.9 List virtualization

Q258. Scenario: a component works but shows stale data after an update on another screen. Why? Typical causes: data copied into local state from props (doesn't follow later changes); a client cache not invalidated after a mutation (call `invalidateQueries`, revalidate the route, or `revalidatePath`/`updateTag`

in Next.js); a `memo`/selector returning a cached object because state was mutated in place; or two separate copies of the same data in different stores. Keep one source of truth and invalidate after writes. → *More:* §9.10 Mutations, invalidation and optimistic updates · §3.8 Derived state — choosing the minimal state

Q259. ⋆ Scenario: how would you design a large-scale React app (senior question)? Talk through: rendering strategy per route (SSR/SSG/SPA, Server Components); routing and code splitting by route; state layers (server state cache, URL state, local state, a small global store); a design system with accessible primitives; feature-based folders with clear boundaries; type safety end-to-end (TypeScript, schema validation); testing pyramid (component tests + E2E on critical flows); performance budgets and Web Vitals monitoring; error boundaries + logging (Sentry); feature flags and CI/CD. → *More:* §14.11 Folder structure · §14.12 Design systems · §13.11 Choosing a framework

Q260. Scenario: how would you migrate a large app from React 18 to 19 (or from classes to Hooks)? Upgrade to React 18.3 first and fix its deprecation warnings; run the React 19 codemods (`ReactDOM.render` → `createRoot`, string refs, `propTypes`, `act` import); update `@types/react` and fix type errors (`useRef` argument, `JSX` namespace); update libraries that rely on removed internals; run the test suite and StrictMode in development. For classes → Hooks, migrate **incrementally**, leaf components first, behind tests — both styles work together, and error boundaries stay as classes. → *More:* §11.1 Setup and how type-checking fits in

Q261. How do you debug a React application? React DevTools (Components tab: inspect props/state/hooks, find which component rendered an element, "why did this render"; Profiler tab), browser DevTools (breakpoints, `debugger`, Network, Performance), StrictMode and lint warnings, error boundaries with `componentStack` logging, React 19's improved error messages (hydration diffs), and reproducing the bug in a failing test first. → *More:* §10.3 Measure first: DevTools Profiler and `<Profiler>` · §16.18 Common errors and fixes

Q262. What is the difference between React and Angular (or Vue)? React is a UI library with JSX, one-way data flow and a large, unopinionated ecosystem (you pick router, state, forms). **Angular** is a full framework (TypeScript, dependency injection, RxJS, its own router, forms and HTTP client, CLI, templates with two-way binding, Signals for reactivity) — more structure, steeper learning curve. **Vue** uses HTML-like templates and fine-grained reactive state, with an official router and store; somewhere between the two. Choose by team experience and how much convention you want. → *More:* §1.1 What React is

---

Sources of the question lists

Question wording and frequency were gathered from these lists and posts (titles only; all answers above are written for this book):

- sudheerj — *reactjs-interview-questions* (GitHub, "List of top 500 ReactJS Interview Questions & Answers", including its Hooks, React 18/19 and Redux sections) — github.com/sudheerj/reactjs-interview-questions
- GreatFrontEnd — *top-reactjs-interview-questions* (GitHub) and blog posts "50 Essential React.js Interview Questions by Ex-Interviewers", "30 Essential React Hooks Interview Questions You Must Know", "30 Basic to Advanced React Interview Questions with Solutions", "Frontend LLD Interview Guide: React machine coding questions" — greatfrontend.com
- h5bp — *Front-end-Developer-Interview-Questions* (GitHub)
- Harendra-jagariya — *frontend-machine-coding-interview-questions-reactjs* (GitHub: counter, countdown, accordion, todo, infinite scroll, pagination, autocomplete, star rating, carousel, toast, tabs, modal, OTP input, progress bar, multi-step form, folder tree, nested comments)
- NamasteDev — "Top machine coding questions for frontend interviews" (parts 1 and 2)
- Peerlist — "80 frontend interview questions asked in most interviews"
- techinterview.org — "Build a Comments Thread: Nested Replies, Pagination, Optimistic Updates", "Machine-Coding: Build an Infinite Scroll List with Virtualization"
- Hyring — "Top 62 React Interview Questions and Answers"; Coursesity — "React Interview Questions: 50 Most Asked"

- react.dev — the React 19, React 19.2 and React Compiler v1.0 release posts (used to check version-specific facts, not as a question list)